BuildScan 3D: Mobile-to-Cloud 3D Gaussian Splatting for Metric Indoor Reconstruction

Project Team

Muhammad Umair 23I-0662
Zaid Fareed 23I-0829
Muhammad Tayyab 23I-0636

Session 2023-2027

Supervised by

Mr. Furqan Nasir



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Contents

Students' Submission i

Anti-Plagiarism Statement ii

Authors' Declaration iii

Abstract iv

Acknowledgments v

Executive Summary vi

List of Abbreviations viii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 2
1.6 Project Scope . . . 3
1.7 Project Modules . . . 3
1.8 Stakeholders and User Classes . . . 4
1.9 Success Criteria . . . 4
1.10 Constraints and Assumptions . . . 5
1.11 Ethical, Legal and DEI Considerations . . . 5

2 Research and Existing Solutions 6

2.1 Research Method / Approach . . . 6
2.2 Existing Solutions / Products . . . 6
2.3 Comparison of Existing Solutions . . . 7
2.4 Limitations and Gaps . . . 8
2.5 Proposed Solution . . . 8
2.6 Complex Computing Problem . . . 9

2.7 Technical Challenges 9
2.8 Research Findings and Design Decisions 10
2.9 Techniques Explored and Parked 11

3 Proposal Feedback and Iteration Plan **13**
3.1 Proposal Feedback Summary 13
3.2 Proposal Action Register 13
3.3 Iteration Strategy 14
3.4 Iteration 1 Plan 14
3.5 Iteration 2 Plan 15
3.6 Work Breakdown and Responsibilities 15
3.7 Progress Against Plan 15
3.8 Delays, Deviations and Recovery Plan 16

4 Software Requirements **18**
4.1 User Interactions / Use Cases 18
4.2 Use Case Diagram 18
4.3 Detailed Use Cases 19
4.4 Functional Requirements 20
4.5 Non-Functional Requirements 22
4.6 Acceptance Criteria 23

5 System Design **24**
5.1 System Architecture 24
5.2 Architectural Components 24
5.3 Interfaces and Dependencies 26
5.4 Architecture Decisions and Rationale 27
5.5 Design Evolution 27
5.6 Domain Model 28
5.7 Data Design 28
5.8 Process / Behavior Design 29

6 Iteration 1: Design and Implementation **33**
6.1 Iteration Objectives 33
6.2 Selected Use Cases / Features 33
6.3 Design 33
6.4 Implementation 34
6.5 Completed Features / Module 34
6.6 Technical Implementation Details 35
6.7 Algorithms and Technical Decisions 37
6.8 Tools, Frameworks and Technologies 37

6.9 Demonstration Evidence . . . 37
6.10 Contributions and Novelty . . . 38
6.11 Iteration Outcome . . . 39

7 Testing and Early Validation **40**
7.1 Testing Strategy . . . 40
7.2 Test Levels and Techniques . . . 41
7.3 Test Cases . . . 41
7.4 Test Results . . . 45
7.5 Acceptance Testing / Validation . . . 46
7.6 Edge and Failure Cases . . . 47
7.7 Defects and Corrective Actions . . . 48
7.8 Current Validation Status . . . 49

8 Project Management and Development Evidence **50**
8.1 Work Completed . . . 50
8.2 Work Remaining . . . 50
8.3 Individual Responsibilities . . . 51
8.4 Contribution of Each Team Member . . . 51
8.5 Repository and Version Control Evidence . . . 52
8.6 Milestones and Commit Evidence . . . 54
8.7 Tools and Tool-Assisted Development . . . 54
8.8 External Components and Dependencies . . . 56
8.9 Risks and Mitigation . . . 56
8.10 Security and Privacy Considerations . . . 57
8.11 Current Limitations . . . 58

9 Remaining FYP Plan **59**
9.1 Remaining Scope . . . 59
9.2 Remaining Modules and Features . . . 59
9.3 Remaining Iterations . . . 60
9.4 Implementation Plan . . . 61
9.5 Testing and Validation Plan . . . 62
9.6 Deployment Plan . . . 62
9.7 Tentative Timeline . . . 63
9.8 Expected Final Deliverables . . . 63

10 Conclusion **65**

A Proposal Action Register **67**

B Detailed Requirements **70**

B.1 Extended Functional Requirements 70
B.1.1 Mobile Capture Module — Detailed 70
B.1.2 Processing Server Module — Detailed 71
B.1.3 Reconstruction Pipeline Module — Detailed 71
B.2 Extended Non-Functional Requirements 72
B.3 Full Traceability Matrix 72

C Detailed Test Cases **74**
C.1 Automated test files 74
C.2 Test cases 76

D Repository and Contribution Evidence **80**
D.1 Repository Structure 80
D.2 Commit History Summary 81
D.3 Repository Overview 81
D.4 Branch Strategy 81
D.5 CI/CD Evidence 82
D.6 Task Tracking 83

E Additional Design Diagrams **85**
E.1 Pipeline Step Sequence 85
E.2 Android App Architecture 86
E.3 Sim3 Alignment 87
E.4 Coverage Map 88
E.5 Domain Model 89
E.6 Capture Application in Use 90

F Meeting and Progress Evidence **92**
F.1 Supervisor Meetings 92
F.2 Progress Log 92
F.3 Problems and Solutions Summary 93

List of Figures

4.1 Use case diagram. The End User is the primary actor; the Processing Server (`server/main.py`) and the Cloud GPU (RunPod pod, or the local GPU) are secondary actors. Take Measurement currently runs in the desktop viewer; the web version is FYP-2 work. 19

5.1 System architecture. Client: Android app (`googleARCore/samples/hello_ar_java/`) and the desktop viewer. Server: `server/main.py`, `pipeline.py`, `runpod_client.py`. Compute: the local GPU, or a RunPod pod reached through the RunPod GraphQL API and SSH/SCP. Both compute paths run the same six steps; `test_server_pipeline.py` checks that the two command lists match. . 25

5.2 Job state machine (`JobStatus` in `server/pipeline.py`). A timeout is enforced per step by the watchdog loop in `run_pipeline()`, which kills the subprocess and marks the job failed; cancel is `POST /jobs/{job_id}/cancel`. In RunPod mode a separate pod watchdog terminates the pod after `MAX_POD_LIFETIME` (default 7200 s). An `uploading` value is defined in `JobStatus` but no code path sets it. 28

5.3 Data flow from the capture files to the model, as run by `server/pipeline.py` (and, step for step, on a RunPod pod). Steps 1–2 read the capture through `common.Capture`; `fallback.py` reads the IMU samples from the MP4's own data tracks (`imu.py`). 30

5.4 Sequence diagram for upload → process → download in RunPod mode (`ProcessingActivity.java`, `server/main.py`, `run_pipeline_on_runpod()` in `server/runpod_client.py`). The app polls every 3 s (5 s after an error) and stops when the status is `completed` or `failed`. The app does not download the model; it is fetched with `GET /download` by the desktop viewer. 32

D.1 Repository root on GitHub. The contributor list confirms the three team members and the GitHub handles used in the commit history: `Umairishuman` (Muhammad Umair), `BlazingBeam` (Muhammad Tayyab) and `Zaid-fareed` (Zaid Fareed). The top-level layout matches Table 8.2. 82

D.2 Branch list with each branch's check status and its ahead/behind count against `master`. Feature branches carry passing checks; the failing status on `master` is the hygiene job described in Section D.5. 83

D.3 GitHub Actions run history. CI passes on the feature branches; the failing runs on `master` are the CD workflow's hygiene job, which rejects a tracked `.env`. That file has since been untracked and ignored, and the key it held must still be rotated (NFR-S2, recorded as not met). 84

D.4 Release tags produced by the CD workflow. Each version tag builds a GitHub Release carrying the signed APK and the web bundle, so every tagged build is reproducible from its commit. 84

E.1 The six pipeline steps and their timeouts (`STEPS` in `server/pipeline.py`). The measured end-to-end time on the development RTX 3070 is ∼75 min; no cloud-GPU time has been measured yet. 86

E.2 Android capture app (`googleARCore/samples/hello_ar_java/.../helloar/`). Arrows from `HelloArActivity` show ownership or navigation (`Intents`); `CaptureLogger` feeds raw depth to `CoverageMap`. `ProcessingActivity` uploads six files plus a zipped `depth/` folder; `camera.csv` and `anchors.csv` stay on the phone. 87

E.3 Sim3 alignment of the COLMAP reconstruction to ARCore's metric frame (`sfm_colmap.py`, export stage; Umeyama fit unit-tested in `test_relocalize_and_sfm.py`). Because ARCore re-anchors silently, only its largest self-consistent segment ends up as inliers. Inlier median residual: about 2.5 cm. The resulting scale is validated against a tape measure (Table 7.6). 88

E.4 Live coverage map (`CoverageMap.java`, `CoverageOverlayView.java`). ARCore raw depth (5 Hz, confidence ≥128, up to 5 m) is accumulated in 10 cm voxels; each voxel records from which of 16 directions it was seen. Visible voxels are drawn as dots, coloured by direction count, with at most 5,000 voxels projected and 2,000 dots drawn, refreshed at 2 Hz. The grid colouring is illustrative. 89

E.5 Domain model. A **Scan** produces exactly one **Dataset**; a Dataset can be submitted as more than one **Job**, because a scan may be reprocessed after a pipeline change; a Job that succeeds produces one **Model**. **Measurement** and the web-facing part of **Structural element** are FYP-2 work, drawn here because the metric scale they depend on is already established by the Sim3 step in Figure E.3. Entity fields follow `Job` in `server/pipeline.py` and the capture files written by `CaptureLogger.java`. 90

E.6 The Android capture app mid-recording on the realme device. The header confirms the capture format (1920×1080 at 30 FPS, FR-M1). The green banner is the live IQA verdict from `QualityMonitor.java`; the row beneath it reports coverage state from `CoverageMap.java` — ceiling and floor not yet seen, and the current voxel observed from 1 of 8 azimuth directions. The red and yellow dots are the coverage overlay drawn by `CoverageOverlayView.java`: red marks surfaces seen from a single direction, yellow from two or three. The elapsed-time counter and the stop control complete the recording UI, and "Searching for surfaces" is ARCore still building its plane estimate early in the session. 91

List of Tables

2.1 Comparison of existing 3D scanning solutions 7
2.2 Techniques explored and parked 11

3.1 Proposal action register 13
3.2 Work breakdown and ownership 16

4.1 Functional requirements with acceptance checks and traceability 20
4.2 Non-functional requirements and how each is measured 22

5.1 Key interfaces between components 26

6.1 IMU fallback on simulated tracking gaps (position error, median / max) . 36
6.2 Tools and technologies used 38

7.1 Test levels and techniques 41
7.2 Automated test files and what each covers 42
7.3 Test cases and their status 43
7.4 Reconstruction quality on the first room: 42 held-out frames that no model was trained on (`fair_compare.py`) 45
7.5 Before and after replacing ARCore poses with image-based (COLMAP) poses 46
7.6 Metric accuracy: model versus tape measure (rigid objects) 47

8.1 Owned modules per team member (from the commit history of each file) . 51
8.2 Repository structure 53
8.3 Key milestones with commit evidence 54
8.4 AI and automated tools used during development 55
8.5 External components and their licenses 56
8.6 Risk register 56

9.1 FYP-2 tentative timeline 63

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *Build-Scan 3D: Mobile-to-Cloud 3D Gaussian Splatting for Metric Indoor Reconstruction*.

Student 1: Muhammad Umair (23I-0662) Signature: _____

Student 2: Zaid Fareed (23I-0829) Signature: _____

Student 3: Muhammad Tayyab (23I-0636) Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Abstract

Current 3D scanning solutions available for indoor locations either need costly LiDAR hardware or complicated step-up procedures that are impractical for construction site personnel, building administrators, and small design companies. The project **BuildScan 3D** is closing the gap by offering a total system capable of converting a footage captured by a usual Android smartphone (such as the realme phone, which has no LiDAR unit or any special equipment) into a metric-scale 3D Gaussian Splatting (3DGS) model, i.e. the one that can be taken for real-world measurements.

The proposed system has five steps: (1) an individual Android app that uses ARCore to record video at the rate of 30 FPS together with camera poses, depth maps, and image quality feedback; (2) a pose-prior stage that relates ARCore OpenGL coordinates to COLMAP/Nerfstudio ones; (3) a cloud-GPU reconstruction stage for which COLMAP Structure-from-Motion with Sim3 alignment of ARCore coordinates has been applied, and training has been performed with the use of Nerfstudio; (4) a structural layer that makes it possible to extract walls, corners, and lengths of walls and doors to create floor plans and measure objects (prototype built); (5) a planned web viewer for measurements and inspecting the model.

At the mid-point evaluation, Iteration 1 stands at *core pipeline complete, refinement ongoing*: the Android capture application processes all the necessary sensor data with live quality and coverage guidance, the FastAPI processing server receives uploads and sends work to RunPod GPU pods as needed, and the top reconstruction (version six) achieved 26.52 dB PSNR/0.8886 SSIM on forty-two held-out frames; tape measure calculations show an average measurement error of 4.8 mm (0.19%) and a maximum error of 0.29% which is far better than the five percent accuracy requirement. The most important technical issues that have been solved already are bridging the tracking gap with IMU, lens self-calibration using COLMAP, and discovering and fixing the problems with irregular Gaussian splat geometry. The open items remaining include refining the 3DGS, creating 3D mesh, completing the structural layer and creating a web workspace (including the floor maps and 1:1 measurements), which should be followed by full-scale field validation.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Mr. Furqan Nasir, for his continuous guidance, constructive feedback, and encouragement throughout this project. His insights on system architecture and research methodology have been invaluable.

We also thank the faculty of the FAST School of Computing, National University of Computer and Emerging Sciences, Islamabad, for providing the academic foundation and resources that made this work possible.

Finally, we acknowledge the open-source communities behind ARCore, COLMAP, Nerfstudio, and 3D Gaussian Splatting, whose publicly available tools and research form the backbone of our reconstruction pipeline.

Executive Summary

BuildScan 3D is a mobile-to-cloud system that allows users to turn a short video of an indoor space captured with a smartphone into an accurate 3D model that can be used to measure.

Problem. The 3D scanning of interiors is usually performed using costly methods that involve either using LiDAR devices which cost more than $10,000 and require multiple preconditions or using photogrammetry, which requires advanced experience in the sphere as well as much time to analyze the data. Many construction companies, facility managers, and students cannot afford to buy the necessary equipment and utilize modern technology to create highly accurate models.

Research. We analyzed the existing developments related to the application of Gaussian Splatting for 3D scanning (1), PocketGS (2), and visual-inertial SLAM (VIGS-SLAM (3)) and found a significant gap. Thus, no system has combined the use of smartphones equipped with ARCore and cloud computing, allowing receiving accurate 3D models in a fully automatic manner.

Solution. Our pipeline includes five stages, which include: (1) capturing video at a rate of 30 fps, ARCore poses, depth and IMU data; (2) alignment of the captures; (3) processing with the use of COLMAP and Nerfstudio; (4) a structural layer (wall planes, corners, wall lengths, door detection) for floor plans and 1:1 measurement, prototype built; (5) viewing the data in a planned web viewer with metric measurements.

Current status (Iteration 1: core pipeline complete, refinement ongoing). This Android capture app is capable of recording all necessary sensor data together with live IQA image quality processing and coverage guidance (using red/yellow/green dot overlay). FastAPI processing server receives the data and dispatches the jobs to RunPod GPU pods that are available as per request (the default pod is NVIDIA RTX 4090, but it can be selected at run-time). The approximate time taken in end-to-end processing using the development RTX 3070 has been calculated at about 75 min; however, the cloud-GPU

timings are yet to be measured, and the under-35-minute RTX 4090 figure is an FYP-2 projection. The current best reconstruction results in 26.52 dB PSNR / 0.8886 SSIM for 42 frames that have not been used for training any of the models so far. Tape-measure accuracy is showing the mean error to be at 4.8 mm (0.19%) with the maximum value of 0.29% (8 mm in the case of 2.8 m wall), which is well inside the target of 5% accuracy. Python code, Android builds, and web UI are all verified via CI/CD process via GitHub Actions. Open items include refinement of 3DGS, creation of 3D mesh, completion of the structural layer, and web workspace.

Risks. The cloud GPU availability and per-job cost (not yet measured); structural layer durability among various rooms (the ceiling-height detection is still not reliable); the success of measurement verification for various room layouts; the unmeasured RTX 4090 processing time. The complete risk register comprises 12 items with probability, effect, and mitigation involved.

FYP-2 plan. Finish up the structural layer and automatic floor plans (Stage 4) and the web-based measurement environment (Stage 5); conduct field trials in further rooms and check correctness of measurement against ground data acquired by means of tape measure. 3D mesh building is also included in the plans yet not scheduled.

List of Abbreviations

3DGS	3D Gaussian Splatting
API	Application Programming Interface
AR	Augmented Reality
ARCore	Google's AR Platform for Android
BA	Bundle Adjustment
CI/CD	Continuous Integration / Continuous Delivery
COLMAP	Structure-from-Motion and Multi-View Stereo Pipeline
dB	Decibels
FPS	Frames Per Second
FR	Functional Requirement
FYP	Final Year Project
GPU	Graphics Processing Unit
IMU	Inertial Measurement Unit
IQA	Image Quality Assessment
LiDAR	Light Detection and Ranging
MVS	Multi-View Stereo
NFR	Non-Functional Requirement
PLY	Polygon File Format
PSNR	Peak Signal-to-Noise Ratio
SCP	Secure Copy Protocol
SfM	Structure from Motion
SIFT	Scale-Invariant Feature Transform
Sim3	Similarity Transform (rotation + translation + scale)
SO(3)	Special Orthogonal Group in 3D
SSH	Secure Shell
SSIM	Structural Similarity Index Measure
TSDF	Truncated Signed Distance Function
VRAM	Video Random Access Memory

Chapter 1

Introduction and Project Vision

1.1 Project Overview

BuildScan 3D is a comprehensive solution that allows anyone to produce photorealistic metric-size 3D models of indoor spaces using just a regular Android phone. The user has to make a short video of walking through the indicated place, while BuildScan uploads the data to the cloud, builds a 3D Gaussian Splatting model, and produces a viewable, measurable digital twin of the scanned area.

The solution is primarily aimed at construction site documentation, facility management and architectural surveying.

1.2 Problem Statement

At this moment, an accurate, metric-grade 3D representation of indoors requires either (i) LiDAR devices costing over $10,000, (ii) photogrammetric methods using professional knowledge and multiple hours of manual post-processing, or (iii) applications for phones (such as Polycam or Luma AI) that will produce models that are visually realistic yet cannot produce 3D models measured in actual units. At present, availability of a fully automated process that would allow for transforming smartphone-based video into a metric-grade 3D model is not at hand. Section 2.4 provides a detailed description of the existing gaps that should be filled.

1.3 Problem Context and Motivation

Construction and facility management are at a loss for time and money due to rework from improper or outdated spatial documents. Small businesses and site crews are in need of rapid and accurate as-built captures for their routine inspections. Modern smartphones equipped with high resolution cameras, IMUs, and processors capable of Augmented Reality (AR), have become potential devices for capturing 3D images. However, software technology should fill the void between consumer AR and professional surveying.

The new development in the field of 3D Gaussian Splatting technology (1) makes it possible to create photorealistic new views of a scene in real time. At the same time, ARCore provides metric-scale camera poses and depth measurement capabilities on Android devices. The combination of this technology with the use of cloud GPU computing will allow for the complete automation of the indoor scanning process.

1.4 Project Vision

BuildScan 3D's mission is to make indoor 3D scanning accessible to everyone by simplifying the process so that it is as easy as making a video on a mobile phone. The technology will: (1) assist the user during the scanning process, ensuring high-quality results; (2) process any recorded video in the cloud; (3) provide a 3D model in a metric scale along with measurement tools; and (4) determine the shape of the room—the wall planes and corners, length of the walls, and the doors—for further automated production of the floor plan along with accurate dimensions on a 1:1 scale.

1.5 Project Objectives

1. Create a capture application for Android that captures videos, ARCore poses, depth maps, and IMU information while also providing live feedback about image quality.

2. Create an automated cloud-GPU processing system to turn recording from the phone into a 3D Gaussian Splatting model, aiming for a total time under 35 minutes on a cloud RTX 4090 (a projection; the current measured processing time is 75 minutes on the development RTX 3070).

3. Meet accuracy requirements of metric scale with less than 5% difference from actual measurement against a tape measure for distance measurements done in the model.

4. Be able to have held-out PSNR of over or equal 25 dB for photometric quality of reconstruction.
5. Develop structural layer that extracts wall planes, corners, wall dimensions and doors from the model for automatic creation of floor plan and metric measuring of the space.
6. Develop 3D viewer that operates online with point to point measuring tools.

1.6 Project Scope

In scope:

- Android capturing tool that has integrated ARCore, real-time IQA, and coverage suggestion.
- Automated reconstruction process which includes extraction of keyframes, COLMAP SfM, Sim3 for metric scaling, and splatfacto 3DGS training.
- Local and cloud-based GPU serving via FastAPI processing server.
- 3D model measurement.
- Structural layer including construction of plane, corners, wall lengths, detection of doors, and making of floor plans (prototype was created in FYP-1; completion in FYP-2).
- Three.js web viewer as model viewing and measurement application (FYP-2).
- 3D mesh production based on the reconstructed model (plan exists, not yet scheduled).
- CI/CD pipeline leveraged through GitHub Actions.

Out of scope: outdoor environments and building-wide scans, iOS (ARKit), real-time on-device reconstruction, and BIM export.

1.7 Project Modules

1. **Mobile Capture Module**: Android application utilizing ARCore technology to conduct video recording, executing camera posture tracking, depth sensing activities, providing IMU telemetry and giving real-time coverage along with quality guidance.

2. **Pose Prior Module**: Coordinate alignment from ARCore OpenGL coordination system to COLMAP or Nerfstudio reference using conversion of quaternion to rotation matrix and Sim3 metric alignment.

3. **Reconstruction Pipeline Module**: COLMAP extracting of features, matching, mapping, undistorting of images, lens self-calibration, and training of Nerfstudio splatfacto 3DGS.

4. **Processing Server Module**: FastAPI backend responsible for uploading datasets, managing jobs, tracking and dispatching of the cloud GPU using RunPod API.

5. **Structural Layer Module**: Recreation of the construction of the room next to the splat model (`structure.py`): the planes of the floors and walls, corners as well as the walls' lengths, doors or any other opening (the floor polygon which is used for providing plans and measurements). Prototype created; finishing shall occur in FYP-2.

6. **Web Viewer Module** (FYP-2): Viewer created in Three.js and WebGPU containing a rendering of the splat and accommodating 1:1 measurements.

1.8 Stakeholders and User Classes

Main users are construction site supervisors, facility maintenance teams, and architectural inspection teams requiring prompt as-built documentation. Other users can be students of architecture and civil engineering, and property managers in real estate. Administrators manage cloud GPU resources and watch over the processing tasks. The supervisor, Mr. Furqan Nasir, carries out the academic assessment and guidance.

1.9 Success Criteria

1. Metric precision: distances taken on the model are within 5% of the tape-measure actual distance (measured: average 0.19%, maximum 0.29%).

2. Reconstruction quality: held-out PSNR ≥ 25 dB, SSIM ≥ 0.85.

3. Processing duration: the total time taken from submitting the data to the visible model is under 35 minutes on the NVIDIA RTX 4090 cloud (FYP-2 goal; projected and not yet measured; the present result is 75 minutes on the development RTX 3070).

4. Capture guidance: the real-time IQA helps users decrease the errors by at least 50% in the process compared to the case when no interactive process is suggested.

5. Structural layer: walls, wall lengths, and doors are extracted automatically with the accuracy being within accepted 5% (FYP-2; not checked against the tape measure yet).

1.10 Constraints and Assumptions

The device should have ARCore compatibility (provided by Google) and that is achieved through a common Android phone such as the realme phone, which lacks any specialized hardware, including LiDAR. The processing relies on the internet, specifically a RunPod account. Make sure that the scanned area is limited to one room (or simple group of rooms)—it must be well lit in order to track the spatial movements.

1.11 Ethical, Legal and DEI Considerations

- **Privacy**: The recorded video may contain recognizable personal identification elements. The data remains on the cloud account of the user alone, without involving any third party. The system does not save or transmit any extra video information apart from what is necessary in the process.

- **Accessibility**: The application has visual support represented as visual quality indication (a capacity of colour-related visual presentation) and haptic facility represented as vibration (reading of level of quality).

- **Environmental**: The computing resource is minimized by terminating cloud GPU pods immediately after processing the data.

- **Tool disclosure**: GenAI tools (Claude Code, GitHub Copilot, ChatGPT) were employed in code development, debugging, documentation and report preparation, including editing of the report. Full disclosure is provided in Chapter 8, Table 8.4.

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

The methodology involved conducting a systematic literature review together with empirical experimentation. Recent articles covering the years 2023–2025 regarding the themes of 3D Gaussian Splatting, on-device 3D reconstruction, and visual-inertial SLAM were reviewed, with the focus on papers that describe systems operating with monocular video or video data captured in smartphones. After that, the prime candidates were implemented and tested on the ARCore-based data. Significant publications are kept in the research papers repository (`docs/Research Papers/`).

2.2 Existing Solutions / Products

Commercial products:

- **Matterport Pro3**: This professional LiDAR scanner is priced over $6000 and can generate metric models accurately but has the disadvantage of needing specific hardware and charging extra fees for each scan carried out in the cloud.
- **Polycam**: A mobile application based on LiDAR (iPhone Pro) or based on photogrammetry. Although it enables quick data collection, it has limited metric accuracy on non-LiDAR devices and does not have an automatic process in the cloud either.
- **Luma AI**: It uses videos to produce a 3DGS that looks fantastic visually but does not have an option to scale and is missing various measuring options.
- **RealityCapture**: The professional software for photogrammetry processes visual

data very impressively but requires skills for the operator to take the photos on their own and to use a GPU for processing pictures.

Research systems:

- **3D Gaussian Splatting** (1): Kerbl et al. in the paper published in 2023 noted that the rendering of the radiance field in real time could take place due to the application of 3D Gaussians. The only prerequisite for this process is that the SfM poses should be computed before doing anything else.
- **PocketGS** (2): Conducting training of 3DGS on devices such as smartphones. The advantage of using dense multi-view stereo initialization is that it provides superior geometry compared to sparse points from Structure from Motion (SfM). However, it has the limitation of being applicable only to short sequences in small scenes.
- **VIGS-SLAM** (3): Visual inertial Gaussian splatting SLAM (simultaneous localization and mapping). It allows for real-time tracking and mapping; however, the obstacle here is the need for custom hardware integration, and hence this system cannot be called a turn-key solution.
- **PGSR** (4): Planar Gaussian splatting method for surface reconstruction. It allows you to achieve a higher level of flatness of walls/floors, but the quality of reflection of the surfaces (mirrors, glossy floors) gets worse as a result.

2.3 Comparison of Existing Solutions

Table 2.1: Comparison of existing 3D scanning solutions

Criterion	Matterport	Polycam	Luma AI	PocketGS	Ours
Hardware cost	$6000+	Phone	Phone	Phone	Phone
Metric scale	Yes	Partial	No	No	Yes
Measurements	Yes	Basic	No	No	Yes
Auto processing	Yes	Yes	Yes	On-device	Cloud
Quality (PSNR)	N/A	Medium	High	High	High
Expert needed	No	No	No	No	No
Capture guidance	No	No	No	No	Yes
Floor plan from scan	Yes	LiDAR only	No	No	Prototype

2.4 Limitations and Gaps

Here are the main drawbacks of the solutions available, which play an important role for us:

1. **Price barrier**: Commercial solutions (Matterport, RealityCapture) require costly requirements for hardware or software licenses.

2. **No metric scale from video**: User apps (Luma AI) and research methods (3DGS, PocketGS) produce visually plausible models but without the metric scale, hence no measurement is possible.

3. **No guidance for capturing**: No system exists that provides real-time feedback on the quality of images, tracking status, and coverage during capture, which results in substantial quality failures.

4. **Manual pipeline**: Research methodologies provide manual workflow steps (choosing a photo, setting up COLMAP, configuring the training), which non-specialists are unable to perform.

5. **No structure from plain video**: 3DGS consumer applications and research have a radiance field but no explicit structure of the room (walls, wall length, doors), thus making it impossible to create floorplans or measure accurately; if floorplans exist, they are only based on LiDAR data or other specialized hardware.

2.5 Proposed Solution

BuildScan 3D fulfills the following functions:

- **ARCore for metric scale**: With help of visual-inertial odometry of the phone it is possible to obtain metric-scale poses. The results are later fine-tuned with the use of COLMAP and corrected with Sim3.

- **Live capture assistance**: Real-time assessment of the image, indicating its blurriness, exposure, and movement speed, along with the voxel-based assessment of coverage, facilitates the obtaining of images.

- **Cloud-based pipeline**: FastAPI allows uploading files while RunPod GPU pod is used for processing, as it controls the process from start to finish without intervention from the user.

- **State-of-the-art rendering**: Nerfstudio splatfacto 3DGS builds photorealistic models, which can be visualized in real-time.

- **Structural layer**: the wall planes, corners, wall lengths and doors are taken from the splat model automatically through the use of `structure.py` (prototype); completion in FYP-2.

2.6 Complex Computing Problem

The complicated problem in computing is: How can a sequence of single-lens mobile phone video frames, with noisy ARCore poses and sparse depth measurements, be turned automatically into a metrically scaled 3D Gaussian Splatting model that is good at giving precise measurements in the real world?

This means accomplishing the following:

- Strong Structure-from-Motion using casual, motion-blurred videos with tracking gaps and silent ARCore re-anchoring events.

- The coordination of scales between COLMAP's arbitrary-scale coordinate system and ARCore's metric frame using a Sim3 transform with outlier rejection.

- The good initialization of 3D Gaussians that were received from multi-view stereo points in order to avoid any problems with degenerate geometry (zero-thickness splats, opacity collapse).

- The correct setting of GPU resources, managing each step with timeout watchdogs to prevent the outflow of expenses for cloud resources.

2.7 Technical Challenges

1. **ARCore's unnoticed re-anchoring**: During the SLAM loop closure, ARCore continuously corrects the world frame, resulting in significant pose shifts of up to 1 m and 28°. In our first test room, all poses taken in the first minute were off by about 1 m and 28°, consequently, some parts of the room were captured double. But we eliminated this issue by introducing COLMAP poses into the team's workflow, as it utilizes an entirely different non-ARCore frame. The app now logs ARCore anchors, so re-anchoring events are directly detectable. Lost tracking moments are compensated by the inertial sensors (`fallback.py`).

2. **Lens Distortion**: The camera parameters provided by ARCore do not include the values accounting for lens distortions. Using COLMAP self-calibration, we computed radial distortions ($k_1 \approx 0.09$, $k_2 \approx$ -0.22) critical for achieving sub-pixel accuracy.

3. **Degenerative Gaussian splats**: Our initial training recipe produced zero-thickness, semi-transparent splats ("cotton-like essence") as a result of a flattening regularizer with adverse effects on Adam's gradient normalization. A detailed root-cause analysis (described in `Training_Analysis.md`) allowed us to use Nerfstudio's splatfacto, along with measured-point initialization.

4. **Plain or untextured surfaces**: The presence of white walls and ceilings leads to losses in tracking and unreliable depth estimations. The solution is represented by the introduction of IQA warnings with texture awareness and capturing suggestions for the ceilings and the floors.

5. **Limitations of GPU memory**: With the RTX 3070 (8 GB VRAM), the splat count can reach a maximum of roughly 3 million at full resolution. This was dealt with by training on a 24 GB cloud GPU based on the RTX 4090 through RunPod and optimizing memory utilization (images in pinned CPU RAM).

2.8 Research Findings and Design Decisions

The main research conclusions that guided our development:

1. The poses of SfM were superior to those of ARCore for 3DGS. When examining ARCore poses, their error by median reprojection, in pixels, was equal to 2.32 px; after bundle adjustment, the error was reduced to 0.62 px (626k observations), however, nonetheless, the error carried by re-anchoring in ARCore could not be corrected, nor can the lack of a lens model in ARCore be overcome. Switching to image-based COLMAP poses with lens self-calibration raised the training-view fit from 21.9 dB (12–30 px misaligned) to 28 dB (2–3 px), raised held-out SSIM from 0.78 to 0.86 (0.91 on a second room), and cut the surfaces placed with high errors from 7%–23% to 3% (1% on the second room). The most successful model trained with the earlier poses was our own trainer with COLMAP depth (`gs_colmap_nc`, 22.67 dB / 0.782 held-out). Decided: use COLMAP SfM for training poses, with ARCore providing metric scale through Sim3 alignment.

2. An important point about dense initialization is made by PocketGS (2), which shows that dense multi-view stereo points bring better reconstruction quality (SSIM 0.834 versus 0.791) in comparison to those obtained through sparse SfM points.

The decision made is to begin the process from MVS-measured points that have gone through voxel downsampling to 1 cm.

3. Regularization may introduce adverse effects. Our various loss metrics namely the flattening loss, the depth loss, or the pose refinement, performed worse when used together. In terms of performance, the splatfacto baseline recipe produced best results. The splatfacto baseline achieved 27.81 dB held-out in a different scene (`roomAttemptSadness2`), compared to 24.70 dB achieved by our own trainer with the corrected recipe. Conclusion: The best results can be obtained with standard splatfacto. The custom preprocessing needs to be kept.

4. Gyro-based frame selection leads to improved image sharpness. The motion blur in the frame is determined based on IMU angular velocity data. It has been determined that the PSNR increased by 0.26 dB. *Decision*: Implemented `select_sharp.py` using the recorded per-frame exposure metadata.

2.9 Techniques Explored and Parked

Prior to utilizing image-based poses, several methods were used by the team in an attempt to solve the issue of the blurred "cottony" models that appeared as floating blotches. None of the methods managed to resolve the root of the problem (ARCore's silent re-anchor and missing lens model; finding 1), and even the best model from this era showed results equal to only 22.67 dB / 0.782 on the validation data. These methods were noted here just for historical purposes. All numbers come from the `docs/progress/`.

Table 2.2: Techniques explored and parked

Technique	**What we tried and measured**	**Outcome**
Own GPU bundle adjustment (`ba.py`)	Levenberg–Marquardt with the Schur complement accumulated on the GPU, ARCore as priors on relative and absolute pose. Median reprojection error 2.32 px $\rightarrow$ 0.61 px; trajectory length 0.998$\times$ ARCore.	Parked. Because it starts from ARCore and keeps ARCore priors, it kept the $\sim$1 m / 28° re-anchored segment. Replaced by COLMAP SfM.
Metric3D monocular depth	Single-image metric depth as a dense prior. Per-frame scale varied 0.85$\times$–1.50$\times$ against ARCore points; walls came out wavy. A smooth log-depth correction cut its error against MVS from 2.4% to 1.1%.	Parked. Its errors are regional and consistent across neighbouring frames, so they passed consistency checks and produced floating copies of furniture.

Technique	What we tried and measured	Outcome
Own stereo matcher (`mvs.py`)	GPU plane-sweep MVS: 5 source views, 64 depth hypotheses, 7×7 NCC, forward-backward consistency check. Only about 11% of pixels per frame were measured, because of plain walls.	Parked. COLMAP dense MVS is used for initialisation instead (v5, v6).
Depth supervision	Depth loss towards a fused surface that included single-image fill. Where stereo measured the true depth, the target was off by >5% on ∼30% of pixels and often lay behind the real surface.	Rejected. It fought the photometric loss and was one of the six causes of degenerate splats.
Floater pruning (`clean_splats.py`)	Delete splats contradicted by measured depth. Removed 1.4% of splats for −0.3 dB and no visible change; with filled depth as evidence it removed 9.8% but deleted real surfaces.	Rejected.
Median filtering	Hole-aware 5–9 px median on stereo depth: outliers (>25% error) 3.3% → 2.4–2.7%, but coverage was lost, and it does nothing for the Metric3D fill or for COLMAP depth.	Parked; kept only as an optional step in `colmap_mvs.py`.
Post-hoc flat-forcing (`planarize.py`)	Snapping walls and floor flat after training.	Rejected: it destroyed the glossy floor's reflections and crushed door frames. The structural layer is built next to the splat model instead.

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

During the proposal defense, the committee requested the group to explain the differences between BuildScan 3D and consumer scan applications (Polycam, Luma AI), show metric accuracy as compared to the real measurements, set precise acceptance criteria for the quality of reconstruction, and concentrate on the metrics and the automated process, not addressing the rich UI. All the details of the requests and actions taken are summarized in the action register.

3.2 Proposal Action Register

Table 3.1 records the feedback received at the proposal defense and the action taken on each point.

Table 3.1: Proposal action register

No.	Panel feedback	Action taken	Status	Evidence
1	How does this differ from Polycam or Luma AI?	Added metric scale via ARCore Sim3 alignment (tape-measure error: mean 0.19%, worst 0.29%), live IQA capture guidance and measurement tools; added a comparison table.	Done	Table 2.1

No.	Panel feedback	Action taken	Status	Evidence
2	Show quantitative accuracy, not just visual quality	Tape-measure comparison on four rigid measurements: mean error 4.8 mm (0.19%), worst case 0.29%, against a 5% target. PSNR/SSIM reported on 42 held-out frames.	Done	Tables 7.4, 7.6
3	Define measurable acceptance criteria	PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85, measurement error $\leq$ 5% of tape measure, processing $<$ 35 min on RTX 4090 (projected FYP-2 target; 75 min measured on RTX 3070).	Done	Section 1.9, Section 4.6
4	The pipeline should be automated, not manual	FastAPI server with RunPod cloud GPU integration; one tap in the Android app uploads the dataset and starts the whole pipeline.	Done	Section 6.5
5	Consider cost implications of cloud GPU	Per-step timeouts; pod termination in try/finally; GPU type selectable at runtime via `/config` (default RTX 4090). Cost per scan not yet measured.	Done	Table 8.6 (R1, R2)

3.3 Iteration Strategy

We undertake a strategy that comprises two iterations, represented by FYP-1 and FYP-2:

- **Iteration 1** (FYP-1, Weeks 1–8): Implementation of the key model; thus, we create the app, reconstruction pipeline and processing server, integrate cloud GPU, and test everything on the real data. As of the mid-evaluation, the core pipeline is complete and refinement is ongoing. The deliverables include a functioning android app, FastAPI server, Run Pod integration, and accurate 3D reconstruction.
- **Iteration 2** (FYP-2, Weeks 9–16): Performance tuning and enrichment of the functions offered. The deliverables include the reduction of time necessary for processing and improvement of robustness of the pipeline; creation of the web-viewing page based on Three.js, which can also measure; the structural layer with floor plans; and field validation on multiple rooms.

3.4 Iteration 1 Plan

- **Objective**: Create the complete system from mobile phone capture to 3D model.

- **Features**: ARCore capture, including IQA & overlay capabilities; COLMAP SfM together with splatfacto training; FastAPI server with RunPod dispatch; upload interface inside the Android application.
- **Deliverables**: Working Android APK, processing server, cloud GPU integration, verification of reconstruction quality in two rooms.
- **Evidence**: Live demo of mobile phone capture → upload → cloud processing → 3D model display.

3.5 Iteration 2 Plan

- **Aim**: Speed up processing time, enhance pipeline stability, finalize the structural layer, and integrate measuring instruments.
- **Characteristics**: pipeline performance enhancement (goal $<$35 minutes on RTX 4090, projected; 75 minutes measured on RTX 3070), edge case resilient, wall-snapping measurement using the web viewer, and completion of the structural layer; modules and timetable in Chapter 9.
- **Expected results**: Optimized pipeline with lower processing time; web viewer providing measurement tools; structural layer capable of floor plan export; report on field validation for $\geq$5 rooms.
- **Proof**: Processing time benchmarks; measuring accuracy comparison with tape measure on at least 5 rooms.

3.6 Work Breakdown and Responsibilities

3.7 Progress Against Plan

The core-pipeline milestones for Iteration 1 have been accomplished; refining work is still going on:

- Weeks 1–2: ARCore capture app with primitive recording feature has been achieved.
- Weeks 3–4: COLMAP SfM pipeline with ARCore Sim3 alignment has been completed. The main issues leading to low quality of reconstruction (gaps in tracking, distortion of lens, and problems with splats) have been solved.

Table 3.2: Work breakdown and ownership

Task	Owner	Status
ARCore capture app	Tayyab, Umair	Complete
Live IQA + coverage overlay	Tayyab (IQA), Umair and Tayyab (coverage)	Complete
Upload UI (ProcessingActivity)	Umair	Complete
Reconstruction pipeline (SfM)	Tayyab	Core complete; refinement ongoing
3DGS training & analysis	Tayyab; trainer fixes by Zaid	Refinement ongoing
FastAPI processing server	Umair	Complete
RunPod cloud GPU integration	Umair	Complete
CI/CD (GitHub Actions)	Zaid, Umair	Complete
Unit tests (pytest)	Zaid	Complete
Pipeline optimization (FYP-2)	Umair, Tayyab	Planned
Web viewer (FYP-2)	Zaid, Tayyab	Planned
3D mesh creation	Team	Planned; not yet scheduled
Structural layer (`structure.py`)	Tayyab	Prototype; completion FYP-2

- Weeks 5–6: Live IQA has been realized with coverage overlay and a number of improvements in quality (selection of gyro-aware frames, MVS initialization).

- Weeks 7–8: the processing server has been developed and integrated with RunPod, the upload interface has been built, and CI/CD has been finalized.

- **Current issues to be addressed**: 3DGS refinement, 3D mesh generation, finishing the structural layer and the web workspace.

3.8 Delays, Deviations and Recovery Plan

- **Deviation**: At first, we came up with our unique script for training (`train.py`) that included depth supervision, flattening regularization, and pose correction. But after carrying out a thorough analysis (detailed in `Training_Analysis.md`) we realized that it had proved less effective than splatfacto. Consequently, we switched to splatfacto, and kept our unique preprocessing intact. The overall process took around two weeks, but the analysis allowed us to make the right decision about the design.

- **Recovery**: The basic pipeline has been launched, but Iteration 1 is not complete yet: there are still some points left to resolve. Therefore, the FYP-2 project will make use of the one-command v6 pipeline and the processing time decrease, and will make plans for structural layer and web workspace in Iteration 2a-2b. Actual 3D mesh creation is in the scope of the project but is not included into the agenda so far.

Chapter 4

Software Requirements

4.1 User Interactions / Use Cases

The system operates with three main types of interaction of the user:

1. **Capture**: The user starts the application on Android, initiates recording operation, then walks around the room being shown feedback on the recording quality, and stops recording after this and taps "Upload & Process."

2. **Monitor**: The user monitors the processing progress showing the current phase, percentage completed, time elapsed, and logs.

3. **View/Measure**: The user checks the resultant 3D model on the web viewer and can calculate certain distances (FYP-2).

4.2 Use Case Diagram

The implementation of the system consists of three actors (See Figure 4.1): *End User* (perform captures and views), *Processing Server* (executes jobs), and the *Cloud GPU* (carries-out reconstruction). The use cases are Record Scan, Upload Dataset, Monitor Progress, View Model, Take Measurements and Configure Server. Each of them has the relevant code: Record Scan corresponds to `HelloArActivity` and `CaptureLogger`; Upload Dataset and Monitor Progress refer to `ProcessingActivity` and the `/upload` and `/status` endpoints of `server/main.py`; Configure Server links to the settings menu of the app (server URL) and `/config`; View Model and Take Measurements correspond to the desktop viewer today and web workspace in phase FYP-2.

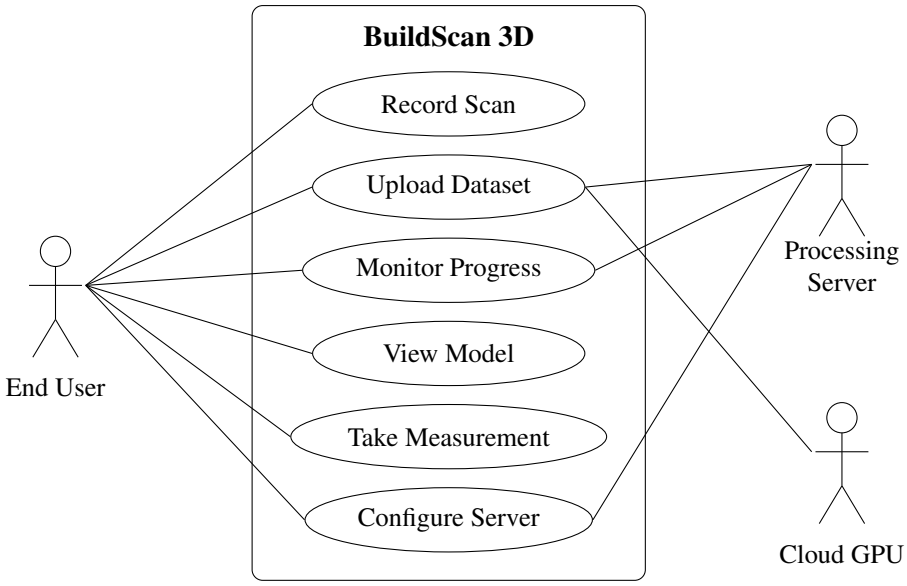


Figure 4.1: Use case diagram. The End User is the primary actor; the Processing Server (`server/main.py`) and the Cloud GPU (RunPod pod, or the local GPU) are secondary actors. Take Measurement currently runs in the desktop viewer; the web version is FYP-2 work.

4.3 Detailed Use Cases

UC-1: Record Indoor Scan

- **Actor**: User
- **Precondition**: Application downloaded on Android device compatible with AR-Core.
- **Flow of Events**: The User starts the application → presses Record → moves around the space → gets live feedback regarding IQA (blurriness, speed, darkness, texture)→ stops the recording → app displays the summary of quality data.
- **Postcondition**: The information is saved on the device (video, poses, depth, quality, meta).

UC-2: Upload and Process

- **Actor**: User
- **Precondition**: Recording is over; URL of the server is configured.

- **Flow of events**: The User clicks on Upload and Process → the app compresses the files and uploads them onto the server → server creates a job and sends it to GPU → app checks progress every 3 seconds communicating step, percentage, logs.
- **Postcondition**: The 3D model (`splat.ply`) is ready for downloading.
- **Exception**: The upload does not go through → button for retrying exists; failure of the process → error message shows the step name.

4.4 Functional Requirements

Table 4.1 lists the functional requirements by module. Each has a priority, a status, a verifiable acceptance check, and its trace to the implementing file and the test case (Chapter 7) that checks it.

Table 4.1: Functional requirements with acceptance checks and traceability

ID	Requirement (dependency)	Pri.	Status	Acceptance check	Implementation; test
Mobile capture module					
FR-M1	Record video at 30 FPS with per-frame camera poses (position, quaternion, intrinsics) in `poses.csv` (ARCore SDK)	Must	Done	One pose row per tracked video frame	`CaptureLogger.java`; TC-1
FR-M2	Log ARCore tracking state and failure reasons in `tracking.csv` (FR-M1)	Must	Done	One row per frame with state and reason	`CaptureLogger.java`; TC-1
FR-M3	Save ARCore feature points with confidence in `points.csv` (ARCore SDK)	Should	Done	Points written with timestamps	`CaptureLogger.java`; TC-1
FR-M4	Save raw depth maps with confidence at 5 Hz in `depth/` (ARCore Depth API)	Should	Done	Depth written at most every 0.2 s, only when ARCore produced a new map	`CaptureLogger.java`; TC-1
FR-M5	Compute and display live IQA: blur, brightness, motion speed, texture (FR-M1)	Must	Done	Warning shown on blurred, dark, fast or textureless frames	`QualityMonitor.java`; TC-2
FR-M6	Show a coverage overlay: red, yellow, green by observed directions (FR-M4)	Must	Done	Dot colour follows the 1 / 2–3 / ≥4 direction rule	`CoverageMap.java`; TC-3

ID	Requirement (dependency)	Pri.	Status	Acceptance check	Implementation; test
FR-M7	Upload the complete dataset by multipart HTTP POST (server available)	Must	Done	Server returns a job ID; files saved	`ProcessingActivity.java`; TC-8
FR-M8	Configure the server URL from the settings menu	Must	Done	URL persists across app restarts	`HelloArActivity.java`; –
FR-M9	Show progress: step, percentage, elapsed time, recent log lines (FR-S5)	Must	Done	Fields update from `/status` every 3 s	`ProcessingActivity.java`; TC-9
Processing server module					
FR-S1	Accept multipart uploads (video, poses, tracking, points, quality, meta, `depth.zip`)	Must	Done	Missing file rejected; corrupt zip fails cleanly	`main.py`; TC-8
FR-S2	Create a unique job ID per upload and track its status	Must	Done	Status has the fields the app reads	`pipeline.py` (`Job`); TC-9
FR-S3	Run the pipeline with per-step timeout watchdogs (pipeline scripts)	Must	Done	Step over its timeout is killed; job failed	`pipeline.py` (`StepConfig`); TC-15, TC-14
FR-S4	Support local GPU and RunPod cloud GPU modes (RunPod API key)	Must	Done	Mode switch via `/config` reaches the right starter	`runpod_client.py`; TC-7, TC-10
FR-S5	Expose a progress-polling endpoint (FR-S2)	Must	Done	`GET /status/{id}`; unknown job 404	`main.py`; TC-9
FR-S6	Provide a download endpoint for `splat.ply` (FR-S3)	Must	Done	404 unknown, 409 unfinished, file when done	`main.py`; TC-16
FR-S7	In RunPod mode, always terminate the pod after processing, even on failure (FR-S4)	Must	Done	`terminate_pod()` on every path where a pod exists	`runpod_client.py`; TC-7
Reconstruction pipeline module					
FR-P1	Extract keyframes by gyro-aware sharpness prediction (exposure metadata)	Must	Partial: validated in v4–v6, not yet in the one-command pipeline	Least predicted blur per 0.2 s window	`prepare.py`, `select_sharp.py`; TC-5
FR-P2	Run COLMAP SfM with SIFT, lens self-calibration and global mapping (COLMAP 4.2)	Must	Done	Frames register; reprojection error <1 px	`sfm_colmap.py`; TC-5, TC-18
FR-P3	Compute a Sim3 alignment from COLMAP to ARCore's metric frame (FR-P2, FR-M1)	Must	Done	Model distances within 5% of tape measure; measured mean 0.19%, worst 0.29% (Table 7.6)	`sfm_colmap.py`; TC-6

ID	Requirement (dependency)	Pri.	Status	Acceptance check	Implementation; test
FR-P4	Train splatfacto and export `splat.ply` (Nerfstudio 1.1.5)	Must	Done	Held-out PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85; v6: 26.52 dB / 0.8886	`pipeline.py`; TC-13
FR-P5	Bridge ARCore tracking gaps with the IMU (FR-M2)	Should	Done	Bridged poses land on both gap anchors	`fallback.py`; TC-4

4.5 Non-Functional Requirements

Table 4.2 lists the non-functional requirements with their measurement method and current status. Several are not yet measured; they are marked as such rather than assumed.

Table 4.2: Non-functional requirements and how each is measured

ID	Category	Requirement	Measurement	Status
NFR-R1	Reliability	The cloud GPU pod shall be terminated within 60 s of job completion or failure (prevents billing leaks).	Server log timestamps on a live pod	Not yet measured
NFR-R2	Reliability	Each pipeline step shall have a timeout watchdog; if exceeded, the step is killed and the job marked failed.	Timeouts in `StepConfig` (TC-15); kill path (TC-14)	Implemented; kill path not yet executed
NFR-U1	Usability	The capture app shall give real-time visual feedback without the user needing to understand 3D reconstruction.	User testing with external users	Not yet executed (planned FYP-2)
NFR-U2	Usability	Processing progress shall show a progress bar, step name and elapsed time.	UI inspection	Implemented
NFR-P1	Performance	End-to-end processing (upload to exported `splat.ply`) shall take under 35 min on an NVIDIA RTX 4090 cloud GPU. FYP-2 target and a projection; the only measured time is $\sim$75 min on the RTX 3070 (matching 23, mapping 6, training 45 min).	Wall-clock time on an RTX 4090 pod	*Outstanding: not yet measured*
NFR-P2	Performance	The app shall keep a 30 FPS camera preview while recording (no frame drops from logging).	No frame drops in logcat over a 3-minute recording	Not yet measured
NFR-P3	Performance	The recording fast-path shall skip non-essential rendering (planes, virtual objects, PBR lighting).	Code inspection; GL thread profiling	Implemented; not profiled

ID	Category	Requirement	Measurement	Status
NFR-S1	Security	The RunPod API key shall be loaded from environment variables or a `.env` file, never hardcoded.	Code review; CI secrets check; `/config` never returns the key (TC-10)	Implemented
NFR-S2	Security	The `.env` file shall be excluded from version control via `.gitignore`.	`.gitignore` inspection	Met since 22 October 2026 (untracked and ignored); the key committed on 18 October is still in the history and must be rotated

4.6 Acceptance Criteria

All acceptance criteria are specified in Tables 4.1 and 4.2. Project's acceptance limits are set as: held-out PSNR $\geq$ 25 dB and SSIM $\geq$ 0.85 (FR-P4; achieved by version 6), difference no more than 5% vs actual measure (FR-P3; achieved with the greatest difference of 0.29%), and processing duration should not exceed 35 min on RTX 4090 (NFR-P1; not measured yet).

Chapter 5

System Design

5.1 System Architecture

The BuildScan 3D application is constructed on a three-level client-server architecture.

1. The client level is represented by a mobile app for Android, which is capable of capturing video with ARCore, uploading it to the server, and displaying the current progress of processing.

2. The server layer contains a FastAPI backend handling job arrangements, orchestration of the pipeline, and handling RunPod workloads.

3. At the last level of computing, the system performs GPU processing by either using the local computer having all the necessary programs installed or using a pod from RunPod service.

In this system, the data is transmitted in a strict sequence: Capture → Upload → Process → Download. The server is acting as a controller managing the GPU and passing the information to the client through regular polling of the server. Figure 5.1 visualizes how every level function and communicates.

5.2 Architectural Components

Android App (`googleARCore/samples/hello_ar_java/`)

- The `HelloArActivity` class is in charge of the AR camera, including the recording controls, the settings menu and the quality overlay.

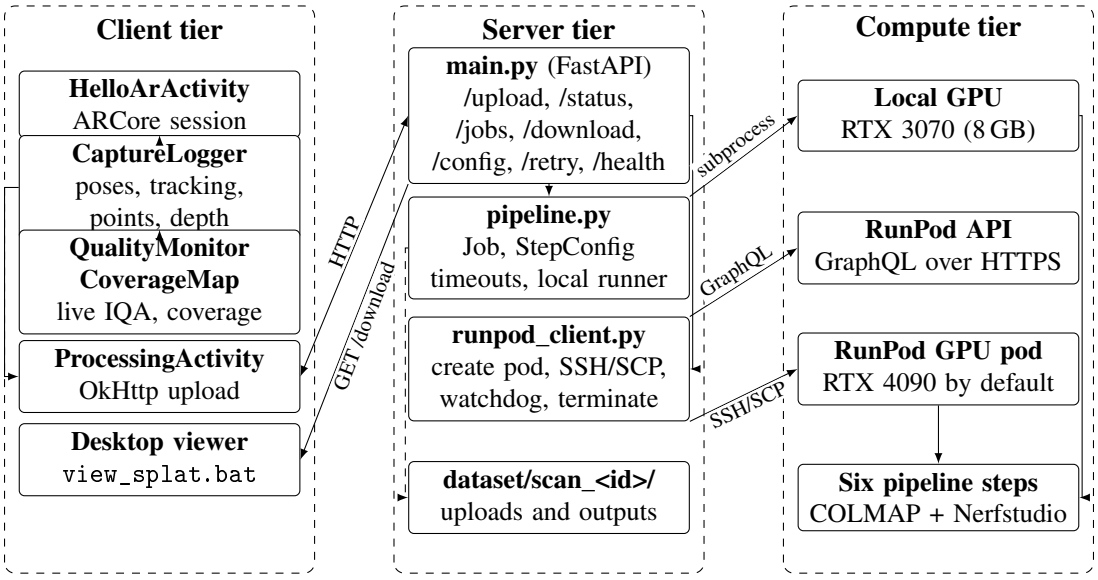


Figure 5.1: System architecture. Client: Android app (`googleARCore/samples/hello_ar_java/`) and the desktop viewer. Server: `server/main.py`, `pipeline.py`, `runpod_client.py`. Compute: the local GPU, or a RunPod pod reached through the RunPod GraphQL API and SSH/SCP. Both compute paths run the same six steps; `test_server_pipeline.py` checks that the two command lists match.

- The `CaptureLogger` class is the background I/O executor which writes the poses, tracking, points, quality, depth and metadata files.
- The `QualityMonitor` class is responsible for the analysis of the camera frames in terms of blurry images, brightness, speed of motion, and texture.
- The `CoverageMap` class keeps track of the coverage by using voxels with 10 cm cubes and maps directions with 8 azimuth $\times$ 2 elevation bins.
- The `CoverageOverlayView` class is responsible for the creation of red/yellow/green dots on the camera images.
- The `ProcessingActivity` class uploads and shows progress with the help of OkHttp.

Figure E.2 illustrates the connection between these classes and screens.

Processing Server (`server/`)

- `main.py`: FastAPI endpoints—`/upload`, `/status/{job_id}`, `/jobs`, `/logs/{job_id}`, `/retry/{job_id}`, `/jobs/{job_id}/cancel`, `/download/{job_id}`, `/config`, `/gpus`, `/datasets`, `/reprocess`, `/run-on-pod`, `/health`.

- `pipeline.py`: Local pipeline runner with `Job` dataclass, `StepConfig` per-step timeouts, and subprocess execution with streaming stdout.
- `runpod_client.py`: RunPod API client (GraphQL pod creation, SSH/SCP helpers, remote pipeline execution, pod watchdog, pod termination in `finally` blocks).

Reconstruction Pipeline (`src/arcore3dgs/`)

- `fallback.py`: IMU-bridged tracking-gap recovery.
- `prepare.py`: Keyframe extraction from video (sharpest frame by Laplacian variance in each window of 8 frames).
- `select_sharp.py`: Gyro-aware keyframe selection (least predicted motion blur per 0.2 s window). Used for the v4–v6 models; not yet part of the one-command pipeline.
- `sfm_colmap.py`: COLMAP SIFT matching + global mapper + lens self-calibration + Sim3 alignment to ARCore metric frame.
- `export_nerfstudio.py`: Convert COLMAP output to Nerfstudio data format.
- Nerfstudio `ns-train splatfacto`: 3DGS training (30k iterations).
- Nerfstudio `ns-export`: Export `splat.ply`.

5.3 Interfaces and Dependencies

Table 5.1: Key interfaces between components

From	To	Interface
Android App	Server	HTTP multipart POST (`/upload`)
Android App	Server	HTTP GET polling (`/status/{id}`)
Server	RunPod API	GraphQL over HTTPS
Server	GPU Pod	SSH/SCP over TCP
Pipeline steps	Next step	Filesystem (dataset directory)
`sfm_colmap.py`	splatfacto	`transforms.json` + images

External dependencies: Google ARCore SDK (Android), COLMAP 4.2 (SfM/MVS), Nerfstudio 1.1.5 (3DGS training), OkHttp 4.12.0 (Android HTTP client), FastAPI + Uvicorn (Python web framework) and the RunPod API (cloud GPU provisioning).

5.4 Architecture Decisions and Rationale

1. **Using COLMAP SfM over ARCore poses for training**: Raw ARCore poses have a 2.32 px median reprojection error (with slow drift); however, using bundle adjustment, the error is reduced to 0.62 px. However, ARCore can re-anchor poses without notifying us (for example, poses before 60 seconds were drifting by 1 m and 28°). ARCore also reports no lens distortion. COLMAP's self-calibrated SfM with lens distortion (k1 $\approx$ 0.09 and k2 $\approx$ -0.22) removes both issues. On the same recording, switching to COLMAP poses made our training approach vastly improve the training-view fit from 21.9 dB to 28 dB and increased the held-out value of SSIM from 0.78 to 0.86.

2. **Using Splatfacto over custom training**: After it became evident that the custom training recipe produces degenerate splats, we opted for Nerfstudio's proven splatfacto implementation. In another room (`roomAttemptSadness2`), we achieved a score of 27.81 dB held-out using splatfacto compared to 24.70 dB of the corrected custom trainer.

3. **Using RunPod on-demand over serverless**: Our on-demand pods enable fully controlled SSH pipeline, and are suited for long processing times (approximately 75 mins in total on RTX 3070).

4. **HTTP polling over WebSockets**: Simpler implementation, works across firewalls/proxies, and 3-second polling interval is sufficient for a pipeline that takes minutes.

5.5 Design Evolution

1. **Proposal** $\rightarrow$ **Mid**: Live IQA and coverage guidance were added (absent from the original proposal). The training switched to COLMAP SfM after realization of ARCore pose accuracy issue. Meanwhile, the GPU bundle adjustment, Metric3D depth, stereo matcher, depth supervision, floater pruning, median filtering, and post-hoc flat-forcing were all tried but shelved by the team (Section 2.9).

2. **Custom trainer** $\rightarrow$ **splatfacto**: After two weeks of analysis resulting in identifying six root-cause issues of the splat degeneracy, training switched to Nerfstudio splatfacto. The custom preprocessing pipeline was preserved and verified.

3. **Local-only** $\rightarrow$ **local + cloud**: RunPod cloud GPU support was added allowing the usage of machines without GPU and achieving speed with GPUs more powerful than the development machine generally used (RTX 4090 as the default one vs RTX 3070, with 8 GB of memory).

5.6 Domain Model

The fundamental entities of the domain comprise the following:

- **Scan**: The session of recording that generates a dataset (video and sensor files).
- **Dataset**: A set of files from one recorded session (video, poses, depth, quality and meta information).
- **Job**: A process performed along with a status (queued, processing, completed and failed) and its current status in progress and output location. Evolution of the job is illustrated in Figure 5.2.
- **Model**: The 3D Gaussian Splatting model (`splat.ply`).
- **Measurement**: A distance measured in the metric model (FYP-2).

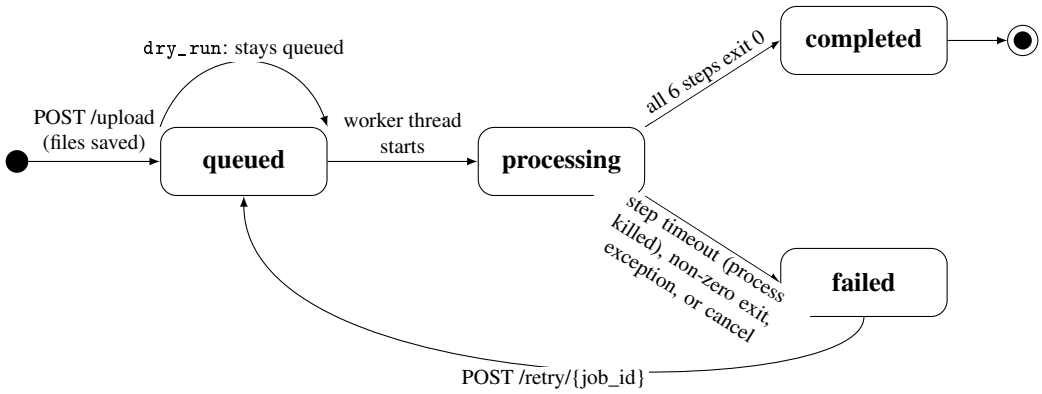


Figure 5.2: Job state machine (`JobStatus` in `server/pipeline.py`). A timeout is enforced per step by the watchdog loop in `run_pipeline()`, which kills the subprocess and marks the job failed; cancel is `POST /jobs/{job_id}/cancel`. In RunPod mode a separate pod watchdog terminates the pod after `MAX_POD_LIFETIME` (default 7200 s). An `uploading` value is defined in `JobStatus` but no code path sets it.

5.7 Data Design

Capture data (per scan):

- `dataset.mp4`: H.264 video, 1920×1080, 30 FPS.
- `poses.csv`: timestamp, tx, ty, tz, qx, qy, qz, qw, fx, fy, cx, cy (per frame).
- `tracking.csv`: timestamp, tracking_state, failure_reason (per frame).

- `points.csv`: timestamp, point_id, x, y, z, confidence (ARCore feature points).
- `quality.csv`: timestamp, sharpness, brightness, motion_speed, texture_level (per analyzed frame).
- `depth/`: uint16 depth maps (mm) + uint8 confidence maps at 5 Hz.
- `meta.json`: device model, camera config, resolution, ARCore version.

Pipeline intermediate data:

- `poses_fused.csv`, `fallback_report.json`: poses with tracking gaps bridged by the IMU.
- `work/images/`, `work/frames.json`: extracted keyframes (JPEG) and their ARCore poses.
- `work_sfm/`: COLMAP database, sparse model, undistorted images, Sim3-aligned `frames.json`, `init_points.ply`.
- `work_sfm/transforms.json`: Nerfstudio-format camera transforms.
- `work_sfm/gs_sfm/splat.ply`: Final 3DGS model.

Figure 5.3 traces the capture files through the six pipeline steps. `quality.csv`, `meta.json` and `depth/` are uploaded and stored with the dataset, but the current six steps do not read them (keyframe extraction runs with `-depth 0`).

5.8 Process / Behavior Design

The procedure for the end-to-end pipeline is as follows:

1. **Fallback** (2-minute timeout): Use the readings from the gyroscope and accelerometer datasets taken from the `dataset.mp4` file. The tracking gaps from `tracking.csv` will be identified and bridged with IMU integration anchored at both ends.
2. **Prepare** (5-minute timeout): Process the given clip, storing the sharpest frame from total 8 frames in terms of the Laplacian variance appearing in the relevant window under script `prepare.py` file. The gyro-aware selection of `select_sharp.py` that produced v6 is not yet applied here.
3. **SfM** (40-minute timeout): COLMAP will be used to perform feature extraction (using the SIFT algorithm), matching of the features, mapping, lens self-calibration, undistorting images, and aligning to the ARCore.

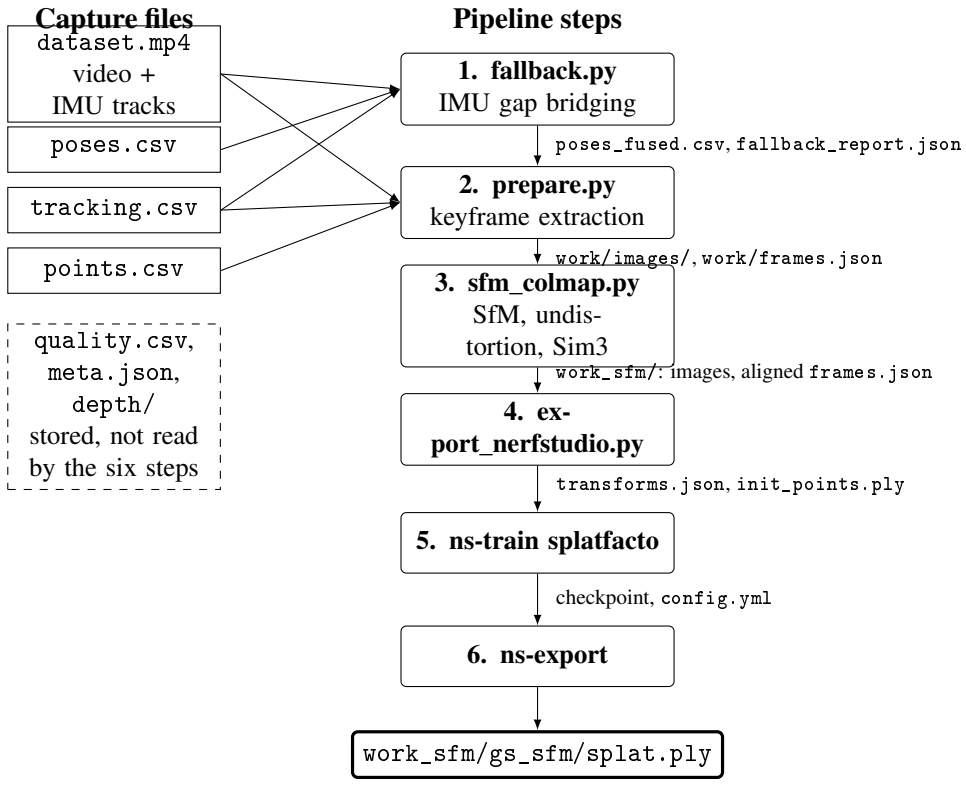


Figure 5.3: Data flow from the capture files to the model, as run by `server/pipeline.py` (and, step for step, on a RunPod pod). Steps 1–2 read the capture through `common.Capture`; `fallback.py` reads the IMU samples from the MP4's own data tracks (`imu.py`).

4. **Exporting** (2-minute timeout): It is necessary to convert the result from COLMAP into Nerfstudio `transforms.json` format.

5. **Train** (60-minute timeout): Train the Nerfstudio splatfacto model for 30,000 iterations with bilateral-grid exposure compensation, scale regularization and fixed poses.

6. **Exporting the model** (5-minute timeout): It is the final stage to create `splat.ply` model.

Figure E.1 depicts the above mentioned steps with the various timeframes taken during each stage.

Every subprocess captures the streaming stdout of the executed step. In case of timeout lapse, the process dies and the job is marked as failed. In RunPod mode, whenever a pod was previously created, it is killed in a `finally` block irrespective of the success of the step. Additionally, the pod is terminated by the watchdog thread when `MAX_POD_LIFETIME` (default value is 7200 s) is reached. Figure 5.4 shows the upload → process → download interaction in RunPod mode.

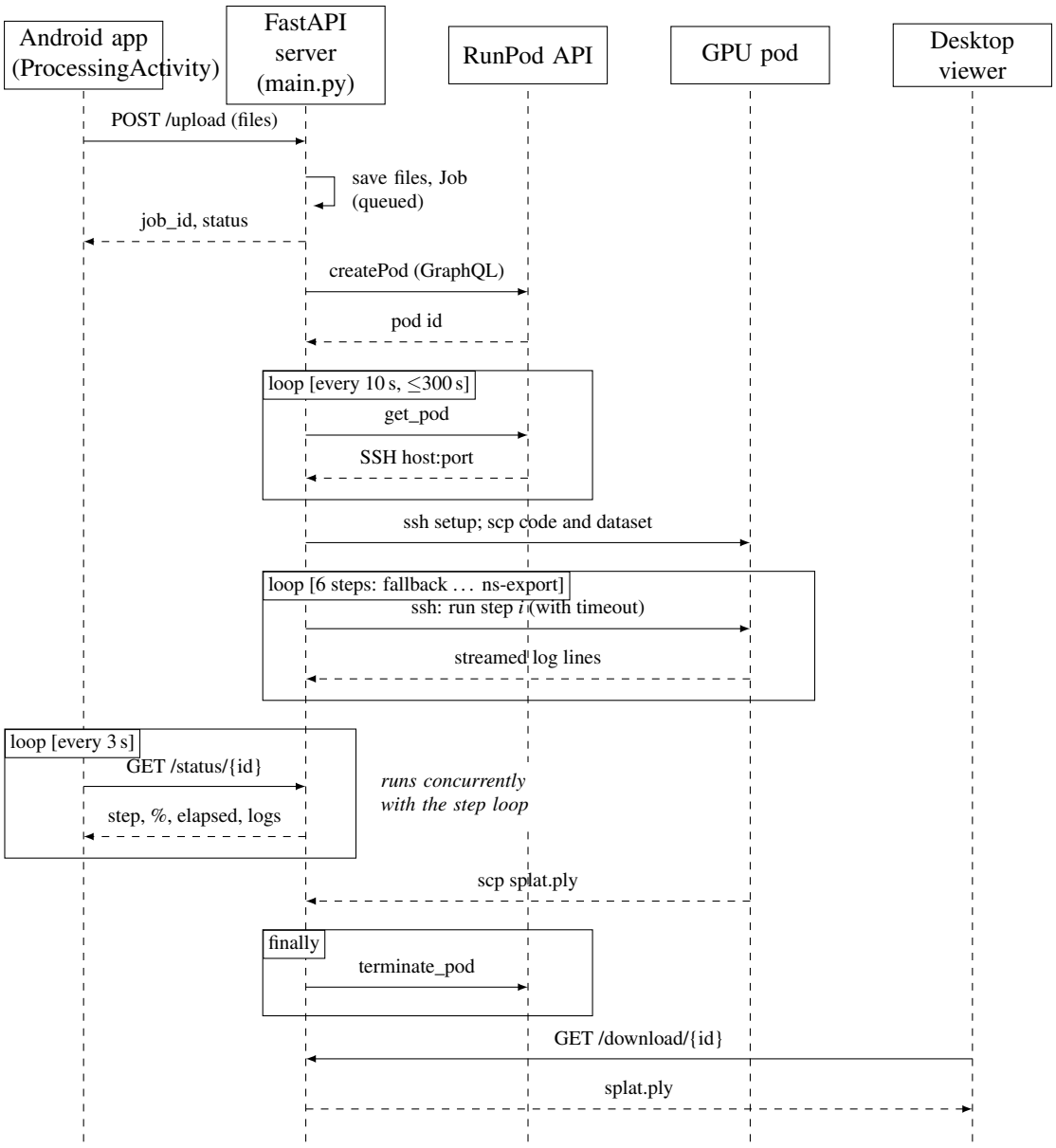


Figure 5.4: Sequence diagram for upload → process → download in RunPod mode (`ProcessingActivity.java`, `server/main.py`, `run_pipeline_on_runpod()` in `server/runpod_client.py`). The app polls every 3 s (5 s after an error) and stops when the status is `completed` or `failed`. The app does not download the model; it is fetched with `GET /download` by the desktop viewer.

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

Build the complete end-to-end pipeline: phone capture → automated upload → cloud GPU reconstruction → metric-scale 3D model. Validate reconstruction quality and metric accuracy on real indoor scans.

6.2 Selected Use Cases / Features

UC-1 Indoor Scan Recording (with live IQA and coverage guidance), UC-2 Upload and Processing (with process monitoring), and UC-3 Viewing of the completed model (for local viewing; web view will be taken up in FYP-2).

6.3 Design

The design of Iteration 1 covered three subsystems:

1. **Capture subsystem**: ARCore session → CaptureLogger (background I/O) → QualityMonitor (frame analysis at a rate of 6 FPS) → CoverageMap (depth → voxels → overlay).

2. **Upload subsystem**: OkHttp multipart POST → FastAPI `/upload` → Job creation → Pipeline dispatch (local or RunPod).

3. **Pipeline subsystem**: consists of six steps in a sequence, whereby each step has a timeout watchdog; all details are provided in Section 5.8 and Figure E.1.

6.4 Implementation

Java (for Android app) was used for the implementation as well as Python (for server and pipeline). The following are the important milestones of the project from realization perspective.

1. **Week 1-2**: The `CaptureLogger` was added to the hello_ar_java sample from ARCore to store the sensor data. The synchronization of pose with video was confirmed (r=0.76-0.79 at the zero offset).

2. **Weeks 3-4**: The COLMAP pipeline was developed (`sfm_colmap.py`) and the coordination aligned (OpenGL→OpenCV) and Sim3 metric alignment was performed as well. The issues related to lens distortion were identified and resolved.

3. **Weeks 5-6**: The degenerate splat geometry was found out and fixed (6 sources of failure are outlined in `Training_Analysis.md`). Gyroscope-based frame selection, live IQA and the coverage overlay were added.

4. **Weeks 7-8**: FastAPI server, RunPod integration with cloud GPU, the Android upload interface (`ProcessingActivity`) and the CI/CD pipeline were all set up and implemented as planned.

6.5 Completed Features / Module

Android Capture App (implemented): The feature allows for recording video at 30 FPS, allowing each frame to be precisely logged regarding its pose, tracking state, the presence of feature points, and the depth, and provides metrics for quality and metadata of devices used for the recording. Live IQA (image quality assessment) can easily show when tracking is lost with the help of a banner that has a red colour, moving too fast is shown in orange, and when video quality is unacceptable by shading the information in yellow. Recording ends with a quality summary dialog and an Upload & Process button that opens `ProcessingActivity`; the server URL is set in the settings menu and persisted in SharedPreferences.

Processing Server (implemented): The endpoints and modes of operation of FastAPI in section 5.2 are switchable at runtime; the step-level watchdog timeout function (Figure E.1); integration with RunPod via pod creation, SSH readiness checking, dependency installation, file upload/download and ending the pod; the API key can be set in the environment or stored in a `.env` file.

Reconstruction Pipeline (core complete, refinement ongoing): The six steps listed in Section 5.8 are completed in one go: IMU gap bridging, keyframe extraction, COLMAP SfM utilizing lens self-calibration and Sim3 alignment (measured using tape with an error of 0.19% on average and 0.29% at maximum), splatfacto learning with exposure compensation using bilateral grids, and exports as `splat.ply`. Gyro-aware frame selection (`select_sharp.py` focusing on exposure metadata) is validated for v4-v6 but not applied in the one-command pipeline yet; the one-command pipeline still uses `prepare.py`'s sharpest frame per window.

CI/CD (implemented): GitHub Actions CI (linting, tests with three versions of Python, web and APK building, repository hygiene), CD has been used for creation of built artifacts and releases as well as a pull request template. Details in Chapter 8.

6.6 Technical Implementation Details

Pose-to-video synchronization (`CaptureLogger.java`): When ARCore records, it records video and poses of the same frames, but poses are captured only during the tracker's `TRACKING` state, meaning that the reported poses contain gaps. During the processing of the frames, we interpolate the poses for each frame's PTS (linear interpolation for position and spherical linear interpolation - slerp - for the rotation) and skip the frames within the gaps which lasted more than 0.12 s.

Sim3 metric alignment (`sfm_colmap.py`): COLMAP's global mapper generates a coordinate system of arbitrary scale. The 7-DOF similarity transform (i.e., rotation + translation + scale) is computed from the matched camera positions of COLMAP against the camera positions of ARCore using RANSAC (2,000 four-point samples, 10-centimeter inlier threshold), followed by a Umeyama refit on inliers, which resulted in an inlier median residual of about 2.5 cm. Since ARCore does the re-anchoring silently, only the largest self-consistent segment ends up being considered an inlier. Any frame that jumps more than 30 cm away from its temporal neighbours is treated as an SfM mis-registration. The resultant metric scale is compared against a tape measure (see Table 7.6). Figure E.3 shows the steps.

Recording fast-path (`HelloArActivity.java`): During recording, the execution of the `onDrawFrame()` method is terminated early just after the rendering of the camera feed and quality overlay (at the rate of 2 Hz). As a result, rendering of all the plane detection, virtual object placement, PBR lighting, work with the cubemaps, and taps is skipped, thus significantly lessening the load on the GL thread.

Featureless-tracking fallback (`imu.py`, `fallback.py`, `relocalize_pnp.py`): If there are just flat walls, ARCore cannot track the position, gets lost, and at the moment of re-tracking can continue operating in terms of the new shifted world frame. However, the fallback processes the gyroscope and accelerometers (201 Hz) recorded by ARCore while recalibrating the IMU-to-camera rotation and sensor offsets. It closes the gaps by using the gyro for rotation and accelerometer integration for positioning with minor cubic correction for both initial and final points of the gap. In case of heading discrepancies at re-tracking stage, the new frame of the map is revealed thus making it possible for the segment following the gap to be aligned with the map by matching its keyframes against the map (PnP).

Table 6.1: IMU fallback on simulated tracking gaps (position error, median / max)

Simulated gap	**IMU fallback**	**Interpolation**
2 s	0.6 / 2.8 cm	5.0 / 12.3 cm
5 s	1.5 / 3.4 cm	20.6 / 52.4 cm
9 s	**3.8** / 8.5 cm	**30.6** / 75.9 cm

In an actual recording, all three heading jumps were successfully detected and re-alignment was able to reduce the error between specified post-gap frames and previous frames from **49.7 px to 8.2 px**. Constraints: Position by IMU can be trusted only if both ends of the gap are anchored and in order to re-align, the user needs to go back into the mapped area (the app warns when tracking failure occurs).

Structural layer (`structure.py`, prototype): The photorealistic splat model remains unaltered. The room shell is built around it instead: floor plane from splat height distribution; the outer wall in each direction selected separately; corners, which represent the intersections of walls, form a floor polygon; textures, with exclusions of the moving reflections; openings classified as doors. In the first room, wall planes of 6.65–6.68 m $\times$ 4.45–4.48 m are confirmed, and 5 doors are detected automatically. The output consists of `structure.json` (planes, corners, openings), which can be used for automatic floor planning and measurement. Ceiling-height detection is not yet reliable and is FYP-2 work.

RunPod lifecycle (`runpod_client.py`): The function `run_pipeline_on_runpod()` encompasses the complete lifecycle of a pod within a try/`finally` clause. It does so through its `finally` clause, which directly invokes `terminate_pod()` whenever a pod is set in motion, and through the enforcement of a hard time limit via a watchdog thread, which ensures that a pod doesn't stay active, even in cases of exceptions, timeouts, or SSH failures.

6.7 Algorithms and Technical Decisions

1. **Gyro-based selection of frames**: For a window of length 0.2 s, we calculate the expected motion blur defined as blur_px = ω × t_exp × f, where ω indicates the angular velocity derived from an inertial measurement unit (IMU) in rad/s, t_exp describes the time of exposure and f refers to the focal length in pixels. The frame that possesses the minimum predicted motion blur shall be selected, which is better than the Laplacian measure of sharpness, whose correlation with the true motion blur is only r = 0.20.

2. **Coverage map**: 3D space is introduced as being represented with the help of voxels 10 cm each side. Then, each voxel is evaluated according to how many from the 16 directions (from 8 azimuthal and 2 elevation angles) where its observation has taken place. The results are represented with dots: those are red in the case of only 1 direction observed or yellow if 2 or 3 directions were seen while green is used in the case of four or more directions. The maximum number of voxels to be processed is limited to 5000 and number of dots is restricted to 2000 to provide reasonable performance.

3. **Finding splat degeneration**: The statistics for each model were calculated (e.g. median values of thickness and width of splats, opacity) and these statistics were compared with respective statistics for the reference implementations of splats. It was found that the interaction of flattening regularizer with Adam method was the primary reason for the creation of zero-thickness splats.

The coverage map is presented in Figure E.4.

6.8 Tools, Frameworks and Technologies

6.9 Demonstration Evidence

It can be shown live:

1. **Android app**: Capture a scan of the room with live IQA feedback and the coverage overlay.

2. **Upload process**: Click on “Upload & Process” and spot progress screen.

3. **Server**: Set up a server via `scripts/start_server.bat` and demo API docs at `/docs`.

Table 6.2: Tools and technologies used

Tool	Version	Purpose
Android Studio	Ladybug	Android app development
ARCore SDK	1.44+	Camera pose tracking, depth
OkHttp	4.12.0	HTTP client for upload
Python	3.10–3.12	Server and pipeline
FastAPI	0.110+	REST API server
COLMAP	4.2	Structure from Motion
Nerfstudio	1.1.5	3DGS training (splatfacto)
PyTorch	2.1+	GPU computation
OpenCV	4.9+	Image processing
GitHub Actions	–	CI/CD
RunPod API	–	Cloud GPU provisioning
ruff	0.6+	Python linting
pytest	8+	Unit testing

4. **3D model**: Show reconstructed `splat.ply` file in local viewer (`scripts/view_splat.bat`).

5. **Metric accuracy**: Measure the dimensions of table and wall and check against the tape-measure values in Table 7.6.

6. **CI/CD**: Show that GitHub Actions are passing in the repository.

6.10 Contributions and Novelty

Summary of all key works based on the available data in this report:

1. Demonstrated issues with commodity ARCore: a silent 1 m / 28° spatial re-localization and the absence of a lens model ($k_1 \approx 0.09$, $k_2 \approx$ -0.22). The present solution is to use image-based poses, keeping ARCore for scale and gravity (Table 7.5).

2. Fallback IMU tracking with no features, verified through simulations: the error was 3.8 cm vs 30.6 cm interpolated for the period of 9 seconds, and post-gap re-alignment which improved from 49.7 px to 8.2 px (Table 6.1).

3. Guidance for the app capture quality in the process of live video capturing obtained: information on blurriness, speed, and darkness of the scene, as well as a vibration-based warning and the overlay of unaccounted areas in the image.

4. **Gyro-based frame selection, bringing measurable results**: improved quality by +0.26 dB /+0.009 SSIM (the largest single gain on the way to v6; Table 7.4).

5. Millimetric accuracy of measurements obtained from the video taken with the phone, without LiDAR: the average deviation = 4.8 mm (0.19%). Worst-case error = 0.29% as compared to tape measurement (Table 7.6).

6.11 Iteration Outcome

The Iteration 1 status is **core pipeline complete, refinement ongoing**. The results obtained thus far are:

- The quality of the reconstruction is well above the required PSNR threshold of 25 dB, with the best model (v6) achieving PSNR of 26.52 dB (0.8886 SSIM) on the 42 frames that were used for testing.
- The tape-measure average error is 4.8 mm (0.19%), with the maximum being 0.29% against the target of 5%.
- The process takes roughly 75 minutes on the local RTX 3070 (matching takes 23 minutes, mapping 6 minutes and training 45 minutes). The target for the FYP-2 project on RTX 4090 is to get that under 35 minutes (the target is a projection and has not yet been measured).
- The application provides real-time monitoring of quality and coverage.
- The server is capable of running both local processing and cloud processing based on the requirements of the application.

The incomplete tasks include (for all tasks apart from mesh creation, please refer to the Remaining FYP Plan chapter):

- **3DGS refinement**: the target is to use v6 recipe for the one-command pipeline and to verify it on the second room.
- **3D mesh creation**: this work is planned but has not yet been scheduled.
- **Structural layer completion**: wall-to-wall dimensions, doors and openings, ceiling-height detection, floor-plan export.
- **Web workspace**: browser viewer with 1:1 measurement.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

Our approach to testing involves utilizing automated unit/integration tests alongside manual validation of the system on real-life indoor scans. This is justified based on the fact that unit tests manage to test only algorithm correctness (coordinate transformations, file processing, API agreements), while complete validation on actual data checks for reconstruction quality and metrics accuracy, which cannot be tested through unit tests.

- **Unit testing**: Python pipeline modules are verified through pytest using synthetic capture folders with no hardware like GPUs and smartphones. The test runs automatically via GitHub Actions CI whenever there are pushes to branches other than master and when making pull requests or through the CD pipeline.
- **Linting**: There is the running of ruff in CI. Real bugs lead to failure of the run, while issues relating to unused code have no impact on running.
- **Integration testing**: There is testing of the server endpoints using FastAPI test client even though the job starters are replaced by recorders and also the client of RunPod even when the network is faked. There are no executions with the real pod of RunPod.
- **Android**: There are no unit tests for Android Java applications. The capture application is verified on the phone, and its packaging is verified via the APK tests.
- **System tests**: Manual execution of the reconstruction pipeline using actual media recordings. Evaluation is conducted by applying the metrics discussed earlier and comparing the outputs of experiments using tape measures. So far, no captured runs of the entire upload $\rightarrow$ server $\rightarrow$ RunPod $\rightarrow$ download path have been made.

- **Regression tests**: Each change in the pipeline is compared against two reference recordings (the first room, `newRoomAttemptSadness`, and the second room, `roomAttemptSadness2`) to determine any regressions in quality. The comparison is done manually using the evaluation scripts, not in CI.

7.2 Test Levels and Techniques

Table 7.1: Test levels and techniques

Level	Scope	Technique	Automation
Unit	Python pipeline modules	pytest on synthetic captures	CI (GitHub Actions)
Static analysis	All Python code	ruff; every file compiles	CI (GitHub Actions)
Integration	Server API, RunPod client	FastAPI test client; faked network	CI (GitHub Actions)
Packaging	Android APK and manifest	APK checker tests	CI (GitHub Actions)
Device	Android capture app	On-device use	Manual; no Java unit tests
System	Capture to model	Pipeline runs on real recordings	Manual
Acceptance	Quality and metric accuracy	Held-out frames; tape measure	Manual

7.3 Test Cases

The pytest suite found in `tests/` includes 12 test files along with a shared `conftest.py` responsible for generating the synthetic ARCore capture folders (without GPU, network, or device). The tests in this suite check only the Python part of the implementation. There are no Java unit tests as the app is tested on an actual device and via the APK/manifest checker test.

Where "Pass (CI)" comes from. The suite was rerun in its entirety on the latest commit (`0d78466`, October 22, 2026) with results equal to 194 passes and 1 skipped (and ruff lint being clean). The only skipped test is the APK signing test that depends on the local Java installation. Previously, on October 12, 2026, GitHub Actions CI run #6 (commit `c3bfcc9` on branch `ci/pipeline-updates`) completed the execution of the same suite on versions 3.10, 3.11, and 3.12 of Python with the "Unit tests" phase being completed

successfully for all three of them. In case of current re-run, only one version of interpreter was used, therefore the evidence of cross-version testing stays within CI. The term “Pass (manual)” denotes a performed run and its data can be found in `docs/progress/`. Other tests are tagged as either “Not yet executed” or “Not yet automated”.

Faults found by that re-run. The retest was not a mere formality: it detected three flaws that were introduced after CI run #6 by the RunPod GPU-selection work. In the method job-cancel, `server/main.py` invoked `time.time()` without importing the `time` library, which caused the error `NameError` in `POST /jobs/job_id/cancel`; the two test files were also out of date with respect to the extended `/config` payload and the new method dealing with GPU name resolution. All three problems were resolved with the help of commit `0d78466`. This is itself evidence for the value of the automated suite and the blocking lint gate.

Table 7.2 shows what each pytest file tests, and Table 7.3 describes every test case with its expected result, evidence and status.

Table 7.2: Automated test files and what each covers

File	Tests	What it checks
`test_fallback.py`	17	`fallback.py` IMU bridging maths on synthetic data: pose loading, tracking-loss intervals, smoothing, gyro integration and bias, bridged gaps that land on the end anchor with the right velocity, rotation blending.
`test_common_fallback.py`	12	`common.Capture` with fallback output: fused poses used when present, bridged intervals marked, interpolated rotations stay valid rotations.
`test_imu.py`	7	`imu.py` protobuf parsers that read gyroscope/accelerometer samples from the ARCore recording, including truncated or garbage input.
`test_common_capture.py`	11	Reading `poses.csv`, `points.csv` and `tracking.csv`: interpolation between samples, no pose inside a tracking gap, intrinsics, frame-time offsets.
`test_common_project.py`	6	OpenGL→OpenCV conversion and pinhole projection (points behind or outside the image rejected).
`test_relocalize_and_sfm6py`		Rigid segment correction (`relocalize.py`) and the Umeyama Sim3 fit used by `sfm_colmap.py`: recovers a known scale/rotation/translation and returns a proper rotation.
`test_process_dataset.py`	3	PLY export and `transforms.json` generation from a synthetic capture.
`test_server_api.py`	19	FastAPI endpoints with the two job starters replaced by recorders: health, config (API key never returned, mode/GPU switching), upload (required files, depth zip, corrupt zip), status, jobs list, download (404/409/200).

File	Tests	What it checks
`test_server_pipeline.py`	12	`server/pipeline.py`: six steps in order, each with a label and positive timeout, iteration count reaches the training command, `Job.to_dict()` fields, log truncation; the RunPod command list matches the local pipeline step for step.
`test_runpod_client.py`	14	`server/runpod_client.py` with network and subprocesses faked: GraphQL queries and errors, pod creation, SSH-port discovery, `wait_for_ready` success and timeout, `terminate_pod`, ssh/scp command construction.
`test_apk_checks.py`	20	The APK/manifest checker used by CI/CD: min SDK, package name, permissions, AR feature, processing screen, signing certificate, APK size; the real manifest passes. This checks packaging, not app logic.
`test_sources_compile.py`	1*	Every Python file in the repository compiles. *Parametrised over every `.py` file.

Table 7.3: Test cases and their status

ID	Test case and expected result	Evidence / input	Level	Status
TC-1	Pose logging completeness (`CaptureLogger.java`). Expected: one pose row per tracked frame with valid position, quaternion and intrinsics	No Android unit test exists; the Python reader of `poses.csv` is covered by `test_common_capture`	Device	Not yet automated
TC-2	IQA blur detection (`QualityMonitor.java`). Expected: blur warning on blurred frames	No Android unit test; no recorded on-device test run	Device	Not yet automated
TC-3	Coverage voxel counting (`CoverageMap.java`). Expected: direction counts drive red/yellow/green dots	No Android unit test; no recorded on-device test run	Device	Not yet automated
TC-4	IMU fallback bridging. Expected: bridged poses land on the end anchor; rotations stay valid; gyro bias removed	`test_fallback`, `test_common_fallback`, `test_imu` (36 tests, synthetic data)	Unit	Pass (CI)
TC-5	COLMAP SfM convergence. Expected: frames register; reprojection error <1.0 px	First room: v3 trained on 574 frames; 1166 gyro-selected frames registered into the v3 model, 1108 kept; global BA 0.97 $\rightarrow$ 0.88 px	System	Pass (manual)
TC-6	Sim3 scale. Expected: known Sim3 recovered; model distances within 5% of tape	`test_relocalize_and_sfm` (Umeyama fit, synthetic data); tape measure: mean 0.19%, worst 0.29% (Table 7.6)	Unit + Accept.	Pass (CI, manual)

ID	Test case and expected result	Evidence / input	Level	Status
TC-7	RunPod client. Expected: correct GraphQL calls, SSH discovery, timeout raised, terminate call sent; on a live pod, termination in every path	`test_runpod_client` with faked network. Live run on a real pod, including termination on failure: not done	Unit / Integ.	Pass (CI); live run not yet executed
TC-8	Upload endpoint. Expected: files saved, job started; bad uploads rejected cleanly	`test_server_api`: multipart upload, missing file, depth zip, corrupt zip	Integ.	Pass (CI)
TC-9	Status polling. Expected: status has the fields the app reads; unknown job is 404	`test_server_api`, `test_server_pipeline`	Integ.	Pass (CI)
TC-10	Config switching. Expected: mode and GPU switch; unknown mode ignored; API key never returned	`test_server_api`: POST then GET `/config`	Integ.	Pass (CI)
TC-11	End-to-end, first room. Expected: `splat.ply` produced; distances within 5% of tape	Capture `newRoomAttemptSadness` $\rightarrow$ local pipeline $\rightarrow$ model; distances vs tape. App upload $\rightarrow$ server $\rightarrow$ RunPod path not yet executed as a recorded test	System	Partial: local pipeline Pass (manual)
TC-12	End-to-end, second room. Expected: model produced; held-out SSIM ≥ 0.85	Capture `roomAttemptSadness2` $\rightarrow$ local pipeline $\rightarrow$ model: SSIM 0.91, 1% badly placed surfaces. Tape measurement not yet done	System	Partial: reconstruction Pass (manual)
TC-13	Reconstruction quality. Expected: PSNR ≥ 25 dB, SSIM ≥ 0.85	v6 on the 42 held-out frames of the first room (`fair_compare.py`): 26.52 dB / 0.8886	Accept.	Pass (manual)
TC-14	Processing timeout. Expected: step killed; job status "failed"; error mentions timeout	Set a step timeout below its run time and run the pipeline. The kill path has never been exercised	Integ.	Not yet executed
TC-15	Pipeline consistency. Expected: six steps in order with positive timeouts; RunPod and local pipelines match; called scripts exist	`test_server_pipeline`	Unit	Pass (CI)
TC-16	Download endpoint. Expected: 404 unknown job; 409 unfinished or failed job; file returned when finished	`test_server_api`	Integ.	Pass (CI)
TC-17	APK and manifest checks. Expected: wrong SDK, package, permission, signer or size reported; real manifest passes	`test_apk_checks`	Unit	Pass (CI)
TC-18	Coordinate conventions. Expected: GL$\rightarrow$CV flips y and z; projection and pose interpolation correct	`test_common_project`, `test_common_capture`	Unit	Pass (CI)

ID	Test case and expected result	Evidence / input	Level	Status
TC-19	Dataset export. Expected: PLY written with header; `transforms.json` built	`test_process_dataset`	Unit	Pass (CI)
TC-20	Source compilation. Expected: every Python file compiles	`test_sources_compile`	Static	Pass (CI)

7.4 Test Results

Automated tests (CI): There exists the pytest suite comprising 128 testing functions across 12 files (one of them parametrised over every Python file, giving 195 collected tests) for which the entire process takes less than three seconds. Out of 20 test cases mentioned earlier in the report, automated tests support 12 (though TC-6 was also performed using the tape measure). In addition, we have two cases (TC-5 and TC-13) performed manually, while two of the cases were tested only partially (TC-11 and TC-12). The remaining three Android test cases did not have automated tests (TC-1 through TC-3), while TC-14 was not executed. Continuous integration is performed every time there is a new commit to the non-`master` branches, as well as during pull requests and from continuous deployment on the `master` branch. The CI workflow itself has not yet run on the current `master` head, so the multi-version evidence remains that of run number 6.

Table 7.4: Reconstruction quality on the first room: 42 held-out frames that no model was trained on (`fair_compare.py`)

Model	Frames	Initialisation	PSNR (dB)	SSIM
v3 (image-based poses)	574	SfM points	26.04	0.8756
v4_sharp (+ gyro-selected frames)	1108	SfM points	26.30	0.8849
v5 (+ dense MVS initialisation)	1108	dense MVS	26.51	0.8879
v6 (+ finer densification)	1108	dense MVS	**26.52**	**0.8886**
Acceptance target			$\geq$25	$\geq$0.85

System-level results: The model with the best results to date, v6, surpasses both benchmarks of acceptance. The factor that produced the biggest single improvement is gyro-aware frame selection (1166 candidate frames examined, 1108 retained), which resulted in a gain of +0.26 dB (increase of SSIM value by 0.009). In turn, dense MVS initialization caused the second step increment equal to +0.21 dB (SSIM improvement of 0.003),

bringing the amount of fused points from 1.15 million to about 460,000 points at 1 cm voxel resolution.

Effect of the pose fix. Table 7.5 demonstrates the performance of the previously employed ARCore-pose pipeline when compared to the current image-based method.

Table 7.5: Before and after replacing ARCore poses with image-based (COLMAP) poses

Measure	Before (ARCore poses)	After (v3)
Training-view fit	21.9 dB, 12–30 px misaligned	∼28 dB, 2–3 px
Held-out SSIM	0.78	0.86 (second room: 0.91)
Badly placed surfaces	7–23%	3% (second room: 1%)

Second room. The image-based pipeline was also applied to a different room (`roomAttemptSadness2`) and the provided result by Muhammad Tayyab showed a held-out SSIM of 0.91 and 1% of poorly positioned surfaces. Meanwhile, the held-out PSNR has not been calculated yet for this room.

Splatfacto versus our own trainer (different scene). For `roomAttemptSadness2`, Nerfstudio's splatfacto trained on our processed data has achieved results of 27.81 dB for held-out views and 28.08 dB for training views compared to 24.70 dB for our own trainer applying the adjusted recipe. This situation has led the developers to migrate to splatfacto. It represents a different scene that has a different protocol than Table 7.4 and is therefore not the score of v6.

7.5 Acceptance Testing / Validation

Acceptance tests compare the system with success criteria as described above:

1. **Metric accuracy**: Four rigid-object distances measured in the model correspond to the tape measure, yielding a mean error of 4.8 mm (0.19%) and a worst case of 0.29% (Table 7.6), against a 5% target.

2. **Reconstruction quality**: V6 produces 26.52 dB of PSNR / 0.8886 of SSIM on the 42 held-out frames from the first room, surpassing the required thresholds of 25 dB / 0.85. The second room reaches a held-out SSIM of 0.91; its PSNR is not

yet measured. Additionally, views rendered from the centre of the room, where no camera stood, are coherent, with the earlier ghosts and floating splats removed.

3. **Processing time**: The total time measured is approximately 75 minutes on the development RTX 3070 graphics card (23 minutes for feature matching, 6 for mapping, 45 minutes for training). The NFR-P1 target of under 35 minutes on the RTX 4090 graphics card is a projection for FYP-2 and has not yet been measured, so NFR-P1 is not yet verified.

4. **Capture usability**: Not yet formally evaluated. Usability testing with external users is planned for FYP-2.

Tape-measure evidence. Table 7.6 compares distances measured in the reconstructed model (first room, image-based pipeline) with a tape measure. Two further measurements on soft edges (a chair and a rug) are recorded in `docs/progress/Problems_and_Solutions.md`; they are excluded from the rigid-object mean because their endpoints are not well defined.

Table 7.6: Metric accuracy: model versus tape measure (rigid objects)

Object	Model (m)	Tape (m)	Error	Relative
Table (short side)	0.458	0.459	−1 mm	0.22%
Table (long side)	1.748	1.750	−2 mm	0.11%
Wall	2.792	2.800	−8 mm	0.29%
Room length	6.692	6.700	−8 mm	0.12%
Mean			**4.8 mm**	**0.19%**

In the worst case, the figure is 0.29%, well below the accuracy goal of 5%. The error does not grow with distance (-8 mm at both 2.8 m and 6.7 m), so the ARCore-derived scale is sound. It is likely due to where the points are clicked in the viewer that there is a small negative offset. Above are four measurements taken in the same room; more rooms are to come in FYP-2.

7.6 Edge and Failure Cases

1. **Loss of tracking while recording**: ARCore gets confused in plain corridors or if it is moved quickly. Hence, the app shows a red notification "Tracking Lost," and stops recording the quality data. The IMU fallback (`fallback.py`) bridges the gap. The positioning error was 3.8 cm for the IMU fallback as opposed to 30.6 cm for interpolation at 9s of simulated gap. The post-realignment error after relocalization is now down to only 8.2 px compared to the original 49.7 px (Table 6.1). The image-based pipeline also registers most frames in the gap according to COLMAP (748

of 759 in the second room). Relocalization would fail if the user never returns to the area that has previously been recorded. Those will be tagged as unverified and hence not used in training. Solution: There is a Live IQA warning that advises the user to slow down, and the coverage overlay shows unscanned areas.

2. **White walls**: Rooms where bare walls have a large area have very few points in the SfM point cloud in these regions. The model will show some blurring or empty spaces. Solution: The IQA "low texture" warning suggests the user captures nearby textured objects.

3. **Mirrors and glass**: Very reflective materials lead to incorrect depth estimates and SfM outliers. *Status*: Known limitation; documented for FYP-2 investigation.

4. **Network Failure during upload**: If the HTTP connection happens to be interrupted during the upload of the file, the `ProcessingActivity` displays an error stating that there has been a failure and provides a retry option. On the server, when an upload is missing a necessary file or has a faulty depth zip, the server cleanly rejects the upload (TC-8). *Status*: Done partially; there has been no end to end testing of the interrupted upload.

5. **RunPod Pod Failure**: If in case the SSH does not come to readiness (pod crash, GPU unavailable), the give-up time set in `wait_for_ready` is 300s while polling every 10s, after which the pod is terminated in the `finally` block and the job is marked failed. *Status*: Implemented; the timeout has been unit tested under faked API conditions (TC-7), while the termination path is yet to be exercised on a live pod.

7.7 Defects and Corrective Actions

ID	Defect	Root Cause	Corrective Action	Status
D-1	Degenerate splats (zero thickness, half-transparent)	Flattening regularizer × Adam interaction; depth loss gradient conflicts	Switched to splatfacto; removed custom regularizers	Fixed
D-2	Blurry models: best held-out result only 22.67 dB / 0.782	ARCore poses used for training (2.32 px raw reprojection error, silent re-anchor, no lens model)	Image-based COLMAP SfM poses with lens self-calibration (held-out SSIM 0.78 → 0.86)	Fixed
D-3	Ghosting artifacts near doorways	ARCore silent re-anchoring events creating pose discontinuities	Training poses from image-based COLMAP SfM; ARCore anchor logging added so re-anchors are detectable	Fixed

D-4	SfM failure on 30 FPS input	Too many similar frames overwhelm COLMAP exhaustive matching	Keyframe selection: one sharp frame per window (`prepare.py`); gyro-aware selection (`select_sharp.py`) for v4–v6	Fixed
D-5	Barrel distortion visible at frame edges	ARCore intrinsics lack lens distortion coefficients	COLMAP self-calibration recovers k_1, k_2; undistort before training	Fixed

7.8 Current Validation Status

Status update on the implementation of acceptance criteria for Iteration 1 (core pipeline completed, refinement ongoing):

- Reconstruction quality: version 6 scores 26.52 dB / 0.8886 for the first room (the requirements are ≥ 25 dB, ≥ 0.85). For the second room, the SSIM results are 0.91; PSNR has not been measured yet.
- Metric accuracy: the average is 0.19%, while in the worst case it is 0.29%, compared to the target of 5% (Table 7.6).
- Processing time: the value is yet to be verified. Previous tests showed that it takes 75 minutes on an RTX 3070; while the target is less than 35 minutes on the RTX 4090 (NFR-P1), which is projected and not yet measured.
- Pod lifecycle: *partly verified*. The `terminate_pod()` function was called from `finally` blocks, and the client side of the solution was tested with faked API, but there were no live tests yet with the pods.
- CI pipeline: all tests were re-run green on the latest versions of the `master` (total of 194 tests were passed and one skipped; all checks were successful); CI run #6 covered Python versions 3.10 to 3.12.
- Not yet automated or executed: Android unit tests (TC-1 to TC-3), processing timeout (TC-14), app-to-RunPod full run.

The remaining validation required for FYP-2 includes testing measurement accuracy in at least three additional rooms that feature different geometrical layouts, accuracy assessment of structural-layer measurements like wall lengths as per a tape measure, doors found versus doors counted manually, web viewer effectiveness test, and stress testing involving large-sized rooms, long duration recordings, and fairly simple Android devices.

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

The Android capturing application has been developed (including ARCore pose, depth and quality logging features, live IQA, coverage overlay and upload interface), the reconstruction mechanism has been implemented (including IMU fallback, key frame extraction, COLMAP SfM with Sim3 metric alignment and Nerfstudio splatfacto training mechanism, model export, with gyro-aware frame selection validated but not yet in the one-command pipeline), the processing server (introducing FastAPI with local and RunPod cloud GPU options and with pod termination function), the CI/CD system (including lint, unit testing, apk and web build and hygiene testing functions), the server launcher (`start_server.bat`, which supports `.env` files), and root-cause investigation of degenerate splat geometry (six causes, documented in `Training_Analysis.md`).

8.2 Work Remaining

Scheduled in Chapter 9: the Three.js/WebGPU Splat viewer with orbit controllers, point-to-point measurement, completion of the structural layer (wall-to-wall dimensions of the walls, doors and openings, ceiling height identification and floor plan export), wall snapping and floor-plan display, improvement of 3DGS (one-click pipeline of v6 version, verification of the second room, motion blur modeling based on IMU); validation in the field is to be done in 3 or more rooms, usability check will involve outside users; 3D mesh creation is in scope as planned work but not yet scheduled.

8.3 Individual Responsibilities

The primary evidence of each member's contribution is the set of modules they own: the files whose commit history carries their name (`git log - <file>`). Table 8.1 lists them.

Table 8.1: Owned modules per team member (from the commit history of each file)

Team Member	Owned modules
Muhammad Umair (23I-0662)	Processing server: `server/main.py`, `pipeline.py`, `runpod_client.py`, `remote_gpu.py`. Android upload and app screens: `ProcessingActivity`, `ScanDetailActivity`, `UpdateChecker`; co-author of `HelloArActivity`, `CoverageMap`, `CoverageOverlayView`, `QualityMonitor`. Web UI prototype (`prototype/`). GPU Docker workflow; co-author of the CD workflow. Server and job scripts (`scripts/`). Proposal, report and project documentation.
Zaid Fareed (23I-0829)	CI/CD: `ci.yml`, `cd.yml`, APK/manifest checker (`.github/android/`), `ruff.toml`, `pytest.ini`, pull request template. The whole pytest suite (`tests/`, all 12 files and `conftest.py`). Trainer experiments and fixes (`train.py` checkpoints and depth loss, `mvs.py`), ARKitScenes trainer experiments (`docs/Experiments_ARKitScenes.md`). CI/CD documentation.
Muhammad Tayyab (23I-0636)	Reconstruction pipeline (`src/arcore3dgs/`): `fallback.py`, `imu.py`, `relocalize_pnp.py`, `prepare.py`, `select_sharp.py`, `ba.py`, `sfm_colmap.py`, `export_nerfstudio.py`, `mvs_init.py`, `structure.py`, evaluation tools (`fair_compare.py`, `fit_diag.py`, `diagnose.py`). Capture logging and live IQA in the app: `CaptureLogger`, `QualityMonitor`; co-author of `HelloArActivity` and the coverage overlay. Training analysis (`Training_Analysis.md`), tape-measure validation and second-room evaluation.

8.4 Contribution of Each Team Member

Muhammad Umair (23I-0662): Project lead. Developed the processing server (FastAPI endpoints, job model with per-step timeouts, local pipeline runner) and RunPod integration (pod creation, SSH readiness, remote pipeline, termination in `finally` blocks). Developed the application's upload and progress screen with the scan gallery/detail screens,

executed the main AR activity and coverage overlay work, created the web UI prototype, and wrote the proposal and a report.

Zaid Fareed (23I-0829): Developed the CI/CD infrastructure (lint, test and build workflows on three versions of Python, Android build and signing checks, repository hygiene, releases) and wrote the entire automated test suite that backs the "Pass (CI)" results in Chapter 7. Provided contributions pertaining to trainer fixes and ARKitScenes trainer experiments.

Muhammad Tayyab (23I-0636): Constructed the reconstruction pipeline: IMU fallback and re-alignment, keyframe and gyro-aware frame selection, bundle adjustment, the COLMAP SfM stages with Sim3 metric alignment, dense MVS initialisation, the structural layer, and the evaluation tools. Wrote the app's capture logging and live IQA, performed training analysis, and produced the v6 model, the tape-measure validation, and the result of the second room.

Commit counts. In total, there are **133** commits divided among four contributors: 85 by Muhammad Umair, 26 by Zaid Fareed, 20 by Muhammad Tayyab (under the alias of "BlazingBeam") and two automated commits from the GitHub Actions bot. These numbers are not an indicator of effort levels by the contributors; rather, they reflect the way each of them commits their changes. For example, Umair commits every little change separately, Tayyab aggregates many changes together, and thus a single commit of his typically represents a significant stage of development. Therefore, the owned modules listed above are more reliable evidence of participation in the project than the number of commits.

8.5 Repository and Version Control Evidence

The GitHub is used to host a project repository that has the following statistics –

- **Repository** – `github.com/Umairishuman/FYP` (private)
- **Branches** – `master` branch (stable) including work branches such as `ci/github-actions`, `ci/pipeline-updates` and `exp/trainer-arkitscenes`
- **Commits history** – 133 commits (85/26/20, including 2 other bot commits)
- **CI** – GitHub Actions runs on work branches, pull requests and (through the CD process) on the `master` branch: ruff lint and pytest in Python versions 3.10, 3.11, and

3.12; Node.js build of the web UI (this is just a placeholder of FYP-2); Gradle APK build checking signing and the manifest; repository hygiene including limitation of files' sizes, secrets checking, and workflow's linting

- **CD** – on a push to `master`, build artifacts; on a version tag, a GitHub release with the APK and web app bundle
- **Code review** – there is a pull request template with area checkboxes as well as testing checklist
- **Task monitoring** – there is `docs/progress/TODO.md`, which is a dated task log of various statuses marked with links to commits and analysis documents

Table 8.2 matches repository layout with the modules mentioned in this report.

Table 8.2: Repository structure

Path	Contents
`googleARCore/samples/hello_ar_java/`	Android capture app; Java sources in `app/src/main/java/.../helloar/`
`HelloArActivity.java`	main AR activity, recording, settings
`CaptureLogger.java`	sensor data recording
`QualityMonitor.java`	live IQA analysis
`CoverageMap.java`, `CoverageOverlayView.java`	voxel coverage tracker and dot overlay
`ProcessingActivity.java`	upload and progress UI
`GalleryActivity.java`, `ScanDetailActivity.java`	saved scans
`src/arcore3dgs/`	reconstruction pipeline
`fallback.py`, `imu.py`, `relocalize_pnp.py`	IMU tracking-gap bridging and post-gap re-alignment
`prepare.py`, `select_sharp.py`	keyframe extraction; gyro-aware frame selection
`sfm_colmap.py`, `export_nerfstudio.py`	COLMAP SfM + Sim3 alignment; Nerfstudio export
`mvs_init.py`, `structure.py`	dense MVS initialisation; structural layer
`fair_compare.py`, `fit_diag.py`	evaluation tools
`server/`	processing server: `main.py`, `pipeline.py`, `runpod_client.py`, `remote_gpu.py`
`.github/workflows/`	CI/CD: `ci.yml`, `cd.yml`, `docker-gpu.yml`
`scripts/`	launchers: `start_server.bat`, `view_splat.bat`, job tools
`tests/`	pytest suite (12 files + `conftest.py`)
`prototype/web-ui/`	web UI prototype
`docs/progress/`	`TODO.md`, `Training_Analysis.md`, `Problems_and_Solutions.md`

Path	Contents
`.env`	RunPod API key and processing mode; ignored and untracked since 22 October 2026 (`.env.example` documents the variables)

8.6 Milestones and Commit Evidence

Table 8.3: Key milestones with commit evidence

Milestone	Description	Evidence
M1: Capture	ARCore recording with pose/depth/IMU logging	Commits by Tayyab, Umair
M2: Pipeline	IMU fallback, COLMAP SfM, splatfacto training	Commits by Tayyab; trainer fixes by Zaid
M3: Quality	Live IQA, coverage overlay, gyro-aware selection	IQA and selection by Tayyab; coverage overlay by Umair and Tayyab
M4: Server	FastAPI + RunPod integration, upload UI	Commits by Umair
M5: CI/CD	GitHub Actions workflows, test suite	Commits by Zaid; workflow merges by Umair
M6: Validation	v3–v6 results, tape measure, second room	Commits by Tayyab; numbers in `docs/progress/`

8.7 Tools and Tool-Assisted Development

IMPORTANT: Generative AI Tool Disclosure

In compliance with university policy, we disclose the use of the following AI-assisted development tools:

Statement of originality: The team developed all the algorithms involved in the project, which includes Sim3 alignment, application of the gyro-aware frame selection method, building coverage maps, IQA methods, and managing the entire RunPod lifecycle. The AI applications were utilized for creating the initial program code, debugging, giving explanations, and making reports, as disclosed in Table 8.4. The rest of material including architecture development, experimental design and data interpretation, and the reasons behind making particular design decisions were produced by the team. The output of AI

Table 8.4: AI and automated tools used during development

Tool	**Usage**	**Scope / Boundaries**
Claude Code (Anthropic)	Agentic coding assistant, used in local and cloud sessions	Code generation and refactoring, debugging and error diagnosis, CI/CD workflow changes (some commits carry a `Co-Authored-By: Claude` trailer), project documentation, drafting of this report, and the correction pass on this report after supervisor review (checking every figure against `docs/progress/` and the team's decisions, rewriting the test and meeting sections, and drawing the TikZ diagrams). All output was reviewed, tested and edited by team members, who remain responsible for it.
GitHub Copilot	Inline code completion	Used for auto-completion in Android Studio and VS Code. All suggestions were reviewed before acceptance.
ChatGPT (OpenAI)	Research queries, algorithm explanations	Used to understand COLMAP parameters, quaternion math, and coordinate system conventions. No code directly copied.

software was well-studied by the members of the team before being integrated into the main system code.

Other development tools: The software tools that were used by the team included Android Studio (Ladybug) with Android Gradle plug-in, Visual Studio Code with Python and Java plug-ins, Git and GitHub, Postman for testing API, ADB for debugging, and NVIDIA Nsight for profiling GPU during training experiments.

8.8 External Components and Dependencies

Table 8.5: External components and their licenses

Component	Version	License	Usage
ARCore SDK	1.44+	Apache 2.0	Pose tracking, depth
COLMAP	4.2	BSD-3	Structure from Motion
Nerfstudio	1.1.5	Apache 2.0	3DGS training
FastAPI	0.110+	MIT	REST API server
OkHttp	4.12.0	Apache 2.0	HTTP client
PyTorch	2.1+	BSD-3	GPU computation
OpenCV	4.9+	Apache 2.0	Image processing

All external components are used under permissive open-source licenses compatible with academic use. No proprietary or restrictively-licensed code is included.

8.9 Risks and Mitigation

Table 8.6 is the project's risk register. Likelihood and impact are rated Low, Medium or High by the team; each risk is handled by the member who owns the affected modules (Table 8.1). The register is reviewed at the start of each FYP-2 iteration.

Table 8.6: Risk register

ID	Risk	Likel.	Impact	Mitigation
R1	Cloud GPU cost exceeds budget	Medium	High	Per-step timeouts, pod watchdog, `terminate_pod()` in `finally`; cost per scan not yet measured (measure in Iteration 2a).
R2	RTX 4090 unavailable on RunPod	Medium	Medium	GPU type selectable at runtime; existing-pod mode; local RTX 3070 fallback (∼75 min).

ID	Risk	Likel.	Impact	Mitigation
R3	Tracking loss on featureless surfaces	High	Medium	Live IQA and coverage overlay; IMU fallback (3.8 vs 30.6 cm over 9 s; 49.7 $\rightarrow$ 8.2 px); unverified segments kept out of training.
R4	Mirrors and glass cause SfM outliers	Medium	Medium	Documented limitation; reflective-region masking in Iteration 2b.
R5	Large rooms exceed GPU memory	Medium	Medium	24 GB RTX 4090 (not yet measured); default densification thresholds; fewer keyframes.
R6	RTX 4090 time unmeasured; $<$35 min target may be missed	High	Medium	Benchmark first in Iteration 2a; speed up matching (23 of 75 min) first; pod watchdog default raised to 2 h.
R7	FYP-2 scope not delivered in time	Medium	High	Fixed order: v6 pipeline, web viewer, structural layer, dashboard (cut first); mesh unscheduled.
R8	Measurement evidence too thin (four measurements, one room)	Medium	Medium	Tape-measure comparisons on $\geq$5 rooms in FYP-2, including structural-layer wall lengths.
R9	Only one phone model tested (realme)	Medium	Medium	Record on a second ARCore phone; exposure and lens metadata logged for comparison.
R10	Nerfstudio/gsplat changes or lost local patch	Low	Medium	Nerfstudio pinned to 1.1.5; pin-memory patch documented; GPU Docker image built by CI.
R11	ARCore API changes	Low	High	ARCore confined to the app; the pipeline reads plain files, so another tracker can feed it.
R12	Team member unavailability	Low	Medium	One shared repository; owned modules and progress logs documented; tests guard the Python code.

8.10 Security and Privacy Considerations

- **API credential management**: The RunPod API credential is loaded at runtime from environment variables or a `.env` file and is never mentioned in the source code directly. Nonetheless, the `.env` file was not in `.gitignore` and was uploaded to the repository on the 18th of October, 2026, contrary to NFR-S2. It was untracked and added to `.gitignore` on the 22nd of October, 2026; the key committed earlier is still in the repository history and must be rotated.

- **CI secret scanning**: The CI hygiene job cannot pass if it keeps a `.env` file or key or certificate documents; also, it scans tracked files for private keys, AWS keys and GitHub tokens (it has no pattern for RunPod keys). The last time CI ran (run #6) was before `.env` was committed; the file has since been untracked, so this rule no longer blocks the next CI run on `master`.

- **Data privacy**: The gathered scanning data is sent to the processing server via a local network. When in production, an HTTPS mode will need to be applied. Collected data can be processed and deleted via the API.

- **Cloud GPU security**: the RunPod pods are formed when they are needed and can only be used in a single operation; no data can be kept on the cloud GPU after.

8.11 Current Limitations

1. **Mirrors and Glass**: The presence of highly reflective surfaces results in SfM outliers with an inaccurate depth model found in those areas. The planned FYP-2 experiment will attempt to detect and mask reflective surfaces.

2. **Large Rooms**: Rooms that exceed roughly 8×8 meters may exceed the GPU capacity limit when training takes place on an RTX 3070 (8 GB). It is expected that a cloud GPU with 24 GB of memory can handle larger scenes. However, this fact has yet to be confirmed.

3. **Outdoor Scenes**: The system has been designed for indoor use; outdoor scenes with sky, vegetation, and long distances are beyond the system capabilities.

4. **Real-Time Processing**: Processing takes 75 minutes on the developing RTX 3070, whereas cloud-GPU timing has yet to be carried out. Real-time processing is not possible using existing 3DGS training times.

5. **Single-Floor Only**: The current coverage map and measurement tools rely on a single-floor environment. Multi-floor buildings require manual editing.

6. **No Web Viewer Yet**: At the moment, 3D models can be viewed only in a desktop program. A web-based viewer based on Three.js will be developed for FYP-2.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

FYP-2 has three main objectives: (1) completing the enhancement of the reconstruction approach and accelerating it; (2) completing the structural layer (wall planes, corners, wall lengths, door identification and floor plans) that enables the photorealistic model to be suitably measured; and (3) developing a web workspace with a 1:1 measurement. The basic capture $\rightarrow$ upload $\rightarrow$ reconstruct pipeline was created during FYP-1 whereas FYP-2 completes and verifies it.

9.2 Remaining Modules and Features

1. **Pipeline Optimization Module**: This involves utilizing the v6 recipe comprising of gyro-selected frames, dense MVS initialization, and finer densification, as the one-command pipeline for validation in the second room. Also, it entails reducing the end-to-end processing time from 75 minutes on the development RTX 3070 to less than 35 minutes on the RTX 4090. The 35-minute benchmark is the projected target instead of a measurement. Techniques employed will include ARCore-pose-guided matching or sequential matching pairs in place of exhaustive COLMAP matching, fewer training iterations and fewer keyframes in slow walks.

2. **Reconstruction Improvement**: This includes IMU-prompted motion blur modeling of the training data and rolling shutter modelling using the per-frame exposure time and rolling shutter information logged by the application. This means that according to the gyroscope, the median training frame is reported as being blurred by between 6 and 12 pixels. In addition, there will be ARCore-started image-only alignment with re-anchor detection based on the logged anchor information, which

could give COLMAP-quality poses faster.

3. **Robustness Improvements**: Enhance the handling of mirrors and glass surfaces (by masking reflective areas), textureless walls, large rooms (>8 m), and different types of lighting conditions.

4. **Structural Layer Module**: Finish the implementation of `structure.py`, which currently creates a room shell next to the splat model: the floor planes and walls, junctions of neighbouring walls (e.g., room dimensions: 6.65-6.68×4.45-4.48 m) and openings (i.e., 5 automatically detected doors). Tasks remaining include measuring wall-to-wall distances, treating doors and other openings as first-class entities, fixing the ceiling height measurement and developing an automatic floor plan generation process.

5. **Web Workspace Module**: A Three.js / WebGPU-based Gaussian splat viewer with orbit/pan/zoom controls. It works in any modern web browser. It can load `splat.ply` with `structure.json` file from structural layer and display the floor plan view.

6. **Measurement Module**: A point-to-point measurement tool capable of measuring distance in metric units within a 3D model. The user clicks two points on either the splat surface or a structural-layer wall, and the system calculates the distance in m in a Sim3-aligned coordinate frame. Target accuracy: within 5% of tape measure (current evidence: mean 0.19%, worst case 0.29%).

7. **Development of 3D Mesh**: Generate a 3D mesh of the room scanned during the reconstruction process in parallel with the creation of the splat model. This task is on the agenda as it was planned, but it is yet to be allocated into an iteration.

8. **Module for Dashboard**: Dashboard accessible via web showing scan history, status of processing, miniature size of models created, and records of measurements.

9.3 Remaining Iterations

- **Iteration 2a** (Weeks 1-4): Optimizing the pipeline and developing the web viewer. The result is a v6 pipeline that requires just one command; processing takes less than 35 minutes on the RTX 4090 (projected, not yet measured); and a browser viewer that allows for point-to-point measurements, validated in a minimum of 3 rooms.

- **Iteration 2b** (Weeks 5-8): Robustness, improving the reconstruction process and structural component. The result is a better handling of edge cases, as well as

adding a structural layer containing the wall lengths, location of the doors, and the floor plan; on the visualization website, it will be possible to snap the measurements to the walls.

- **Iteration 2c** (Weeks 9-12): A dashboard, field testing, and fine-tuning the software. The deliverable is a complete workspace on the web alongside the user testing of the application on at least 5 rooms and the final report.

9.4 Implementation Plan

Iteration 2a: Optimization + Web Viewer (Weeks 1–4)

1. Take the v6 recipe and fit it into the one-command pipeline; substitute exhaustive matching with ARCore-pose-guided or sequential pairs (week 1).
2. Decrease the number of iterations with splatfacto and keyframes for slow walks; test every adjustment in both reference rooms (week 1-2).
3. Create a project in Three.js equipped with a Gaussian splat renderer and camera controls (week 2).
4. Create a measuring device: ray-cast click $\rightarrow$ nearest splat $\rightarrow$ 3D coordinates $\rightarrow$ distance display (week 3).
5. Test measurement accuracy compared to a tape-measure on 3 rooms (week 3-4).
6. Connect the viewer to the FastAPI backend for model loading (week 4).

Iteration 2b: Robustness + Structural Layer (Weeks 5–8)

1. Add mirror/glass reflective-surface detection and masking features (week 5).
2. Experiment with IMU-based motion-blur and rolling-shutter modeling on recordings that support per-frame exposure metadata (week 5-6).
3. Structural layer: fix ceiling height detection; create doors and openings as separate objects with their dimensions (week 6).
4. Structural layer: wall-to-wall dimensions, automate floor-plan exporting from the floor outline (week 7).
5. Use `structure.json` in the web viewer; make measurements snap to walls; add a floor-plan view (week 8).

Iteration 2c: Dashboard + Final Validation (Weeks 9–12)

1. Create the web dashboard: list of scans, processing statuses, model thumbnails (week 9-10).
2. Conduct usability tests with 5+ external users; obtain usability feedback (week 10-11).
3. Resolve usability problems; prepare documentation (week 11-12).
4. Final validation: measurement accuracy, structural-layer accuracy, processing time on 5+ rooms (week 12).

9.5 Testing and Validation Plan

- **Measurement accuracy**: Compare point-to-point measurements in the web viewer against tape-measure ground truth on $\geq$5 rooms of varying size and geometry. Target: error within 5% of tape measure, extending the current four-measurement evidence (mean 0.19%) to more rooms.
- **Structural layer**: For each test room, compare extracted wall lengths against tape measure (target: within 5%) and the detected doors against a manual count.
- **Web viewer performance**: Maintain $\geq$30 FPS on a mid-range laptop (integrated GPU) for models up to 2 million splats.
- **Usability testing**: Task completion rate $\geq$90% for first-time users performing capture, viewing, and measurement tasks.
- **Processing time**: End-to-end pipeline $<$35 min on RTX 4090 (projected target; currently $\sim$75 min measured on RTX 3070). Benchmark each optimization individually.
- **Cross-browser**: Verify viewer on Chrome, Firefox, and Edge.

9.6 Deployment Plan

1. **Server**: Deploy the FastAPI backend and web viewer on a VPS with HTTPS (Let's Encrypt). Static assets served via CDN.
2. **GPU processing**: Continue using RunPod on-demand pods. Evaluate RunPod serverless endpoints if job volume increases.

3. **Android app**: Distribute the APK via GitHub Releases. Google Play Store submission if time permits.

4. **CI/CD**: Extend the existing GitHub Actions to deploy the web frontend on push to master.

9.7 Tentative Timeline

Table 9.1: FYP-2 tentative timeline

Week	Task	Deliverable
1–2	One-command v6 pipeline, matching and training speed-ups + Three.js viewer setup	Processing $<$35 min (projected); viewer MVP
3–4	Measurement tool + accuracy validation on 3 rooms	Measurement accuracy report
5–6	Robustness (mirrors, textureless walls), motion-blur modelling, structural-layer fixes (ceiling height, openings)	Robust pipeline; structural layer v2
7–8	Floor-plan export, wall snapping and floor-plan view in the viewer	Measurable web workspace
9–10	Dashboard + user testing (5+ external users)	Web platform MVP
11–12	Final validation on 5+ rooms, polish, final report	Complete system

9.8 Expected Final Deliverables

1. **Android APK**: The capture app with live IQA, coverage overlay, and uploading functionalities.

2. **Web app**: Gaussian splat viewer with 1:1 measurement tools and floor plan view.

3. **Processing server**: FastAPI with local and cloud GPU compatibility.

4. **Structural component**: per-room `structure.json` (planes, corners, walls length, doors and openings) and floor plan export.

5. **Validation report**: accuracy of measurements taken in at least 5 rooms, structural component accuracy and time measurement statistics.

6. **Documentation**: user manual, API documentation, deployment manual.

7. **Codebase**: complete repository with CI/CD process, tests and documentation.
8. **Final report**: FYP-2 document with complete results and analysis.

Chapter 10

Conclusion

BuildScan 3D takes a simple video recorded with a common Android phone and converts it into a metric-scale 3D Gaussian Splatting model of the room. At mid-evaluation, Iteration 1 stands at *core pipeline complete, refinement ongoing*.

What we found. The design was determined by four major findings: (1) image-based COLMAP poses replace ARCore poses for training: once ARCore's silent re-anchor and missing lens model were removed, the training-view fit increased from 21.9 dB to 28 dB and has improved the held-out SSIM from 0.78 to 0.86; (2) dense MVS initialization contributes to the geometry of splats; (3) excessive regularization brings the quality down when used along with Adam; (4) the choice of frames based on gyrosensor data performs better than the selection based on metrics of sharpness.

What we built and measured. There is a three-tier system that consists of Android capture application with live IQA and coverage overlay, processing server with FastAPI and RunPod dispatch, and COLMAP + splatfacto pipeline with Sim3 metric alignment system working with CI/CD via GitHub Actions. The best current model (v6) has reached 26.52 dB PSNR (target $\geq$ 25 dB) and 0.8886 SSIM (target $\geq$ 0.85) over 42 held-out frames. Tape measure checks give a 4.8 mm average error which is equal to 0.19% over rigid objects with the worst case scenario being 0.29% which is much better than the expected 5% accuracy. Processing takes approximately 75 minutes on RTX 3070 video card while the target of below 35 minutes using an RTX 4090 video card is projected and not yet measured.

Risks and next steps. Twelve risks have been identified and tracked in the risk register (Table 8.6), of which the most prominent are cloud GPU costs (cost per scan not yet measured), the unmeasured RTX 4090 processing time, tracking loss on featureless sur-

faces, and providing the remaining project scope of the FYP-2. Mirrors and glass remain a known limitation. The FYP-2 implies working on 3DGS improvement, the structural layer with automatic floor plans, and the web workspace with 1:1 measurement, validated on more rooms. Moreover, 3D mesh creation is regarded as a planned task that has not been scheduled yet.

Appendix A

Proposal Action Register

The following table records all feedback received during the FYP proposal defense and the actions taken in response.

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
1	How does this differ from Polycam or Luma AI?	Clearly differentiate from consumer apps	Added metric scale via ARCore Sim3 alignment (tape-measure error: mean 0.19%, worst 0.29%), live IQA capture guidance, and measurement tools. Created comparison table.	Done	Chapter 2, Table 2.1

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
2	Show quantitative accuracy, not just visual quality	Provide metric accuracy measurements	Tape-measure comparison on four rigid measurements: mean error 4.8 mm (0.19%), worst case 0.29%, against a 5% target. PSNR/SSIM reported on 42 held-out frames.	Done	Tables 7.4 and 7.6
3	Define measurable acceptance criteria	Set quality thresholds	Acceptance criteria: PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85, measurement error $\leq$ 5% of tape measure, processing $<$ 35 min on RTX 4090 (projected FYP-2 target; 75 min measured on RTX 3070)	Done	Section 1.9, Table 4.3
4	The pipeline should be automated, not manual	Show fully automated pipeline	Built FastAPI server with RunPod cloud GPU integration; one-tap upload from Android app triggers entire pipeline automatically	Done	Chapter 6, Section 6.5

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
5	Consider cost implications of cloud GPU	Address cloud compute costs	Per-step timeouts prevent unbounded billing; guaranteed pod termination in try/finally; GPU type selectable at runtime via `/config` (default RTX 4090)	Done	Risk register, Table 8.6 (R1, R2)

Appendix B

Detailed Requirements

B.1 Extended Functional Requirements

B.1.1 Mobile Capture Module — Detailed

ID	Description	Priority	Dependency	Status
FR-M1	Record video at 30 FPS with per-frame camera poses (position, quaternion, intrinsics)	Must	ARCore SDK	Done
FR-M2	Log tracking state and failure reasons per frame	Must	FR-M1	Done
FR-M3	Save ARCore feature points with confidence	Should	ARCore SDK	Done
FR-M4	Save depth maps at 5 Hz with confidence	Should	ARCore Depth API	Done
FR-M5	Compute and display live blur, brightness, motion, texture metrics	Must	FR-M1	Done
FR-M6	Display voxel-based coverage overlay (red/yellow/green)	Must	FR-M4	Done
FR-M7	Upload complete dataset via multipart HTTP POST	Must	Server available	Done
FR-M8	Configurable server URL via settings menu	Must	–	Done
FR-M9	Display processing progress (step, %, time, logs)	Must	FR-S5	Done

B.1.2 Processing Server Module — Detailed

ID	Description	Priority	Dependency	Status
FR-S1	Accept multipart uploads (video, poses, tracking, points, quality, meta, depth.zip)	Must	–	Done
FR-S2	Create unique job ID and track status	Must	–	Done
FR-S3	Run pipeline with per-step timeout watchdogs	Must	Pipeline scripts	Done
FR-S4	Support local GPU and RunPod cloud GPU modes	Must	RunPod API key	Done
FR-S5	Expose progress-polling endpoint	Must	FR-S2	Done
FR-S6	Provide download endpoint for splat.ply	Must	FR-S3	Done
FR-S7	Always terminate RunPod pod after processing	Must	FR-S4	Done

B.1.3 Reconstruction Pipeline Module — Detailed

ID	Description	Priority	Dependency	Status
FR-P1	Extract keyframes using gyro-aware sharpness prediction	Must	Exposure metadata	Partial: `select_sharp.py` validated (v4–v6), not yet in the one-command pipeline
FR-P2	Run COLMAP SfM with SIFT, self-calibration, global mapper	Must	COLMAP 4.2	Done
FR-P3	Compute Sim3 alignment to ARCore metric frame	Must	FR-P2, FR-M1	Done
FR-P4	Train splatfacto and export splat.ply	Must	Nerfstudio 1.1.5	Done

FR-P5	Bridge tracking gaps using IMU dead reckoning	Should	FR-M2	Done

B.2 Extended Non-Functional Requirements

ID	Category	Description	Measurement
NFR-R1	Reliability	Pod terminated within 60 s of completion/failure	Server log timestamps
NFR-R2	Reliability	Each step has timeout watchdog	Pipeline config file
NFR-U1	Usability	Real-time visual feedback without 3D knowledge	User testing with external users (planned for FYP-2; not yet executed)
NFR-U2	Usability	Progress display with step, %, elapsed time	UI inspection
NFR-P1	Performance	E2E processing $<$ 35 min on RTX 4090 (projected FYP-2 target)	Elapsed time on an RTX 4090 pod; not yet measured (75 min measured on RTX 3070)
NFR-P2	Performance	30 FPS during recording, no drops	logcat frame timing
NFR-P3	Performance	Recording fast-path skips non-essential rendering	GL thread profiling
NFR-S1	Security	API key from env vars, never hardcoded	Code review + CI secrets check
NFR-S2	Security	.env excluded from version control	.gitignore inspection

B.3 Full Traceability Matrix

Req.	Module	Design Component	Implementation File	Test ID
FR-M1	Mobile	CaptureLogger	CaptureLogger.java	TC-1

FR-M2	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M3	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M4	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M5	Mobile	QualityMonitor	QualityMonitor.java	TC-2
FR-M6	Mobile	CoverageMap	CoverageMap.java	TC-3
FR-M7	Mobile	ProcessingActivity	ProcessingActivity.java	TC-8
FR-M8	Mobile	SettingsMenu	HelloArActivity.java	–
FR-M9	Mobile	ProcessingActivity	ProcessingActivity.java	TC-9
FR-S1	Server	/upload endpoint	main.py	TC-8
FR-S2	Server	Job dataclass	pipeline.py	TC-9
FR-S3	Server	StepConfig timeouts	pipeline.py	TC-15, TC-14
FR-S4	Server	RunPodClient	runpod_client.py	TC-7
FR-S5	Server	/status endpoint	main.py	TC-9
FR-S6	Server	/download endpoint	main.py	TC-16
FR-S7	Server	try/finally termination	runpod_client.py	TC-7
FR-P1	Pipeline	prepare.py / select_sharp.py	prepare.py, select_sharp.py	TC-5
FR-P2	Pipeline	sfm_colmap.py	sfm_colmap.py	TC-5, TC-18
FR-P3	Pipeline	Sim3 alignment	sfm_colmap.py	TC-6
FR-P4	Pipeline	ns-train + ns-export	pipeline.py	TC-13
FR-P5	Pipeline	fallback.py	fallback.py	TC-4

Appendix C

Detailed Test Cases

The automated tests are the pytest suite in `tests/`: 12 test files plus a shared `conftest.py` that builds small synthetic ARCore capture folders (no GPU, no network, no phone). The suite tests the Python side of the project only. **There are no Android (Java) unit tests**; the capture app is checked on a phone and, for its packaging, by the APK/manifest checker tests.

Where "Pass (CI)" comes from. The suite was re-run in full against the current `master` head (commit `0d78466`, 22 October 2026): **194 passed, 1 skipped**, with the blocking ruff lint clean. The single skip is the APK signing check, which needs `keytool` from a local Java installation. Earlier, GitHub Actions CI run #6 (12 October 2026, commit `c3bfcc9` on branch `ci/pipeline-updates`) ran the same suite on Python 3.10, 3.11 and 3.12 and the "Unit tests" step succeeded on all three; the re-run above was on a single interpreter, so the cross-version evidence is still CI's. "Pass (manual)" means a recorded run with its numbers in `docs/progress/`. Anything else is marked "Not yet executed" or "Not yet automated".

Faults found by that re-run. The re-run was not a formality: it found three defects introduced after CI run #6 by the RunPod GPU-selection work. `server/main.py` called `time.time()` in the job-cancel endpoint without importing `time`, so `POST /jobs/{job_id}/cancel` raised `NameError` at runtime; two test files had also gone stale against the widened `/config` payload and the new GPU-name resolution step. All three are fixed in commit `0d78466`. This is itself evidence for the value of the automated suite and the blocking lint gate.

C.1 Automated test files

File	Tests	What it checks
test_fallback.py	17	fallback.py IMU bridging maths on synthetic data: pose loading, tracking-loss intervals, smoothing, gyro integration and bias, bridged gaps that land on the end anchor with the right velocity, rotation blending.
test_common_fallback.py	12	common.Capture with fallback output: fused poses used when present, bridged intervals marked, interpolated rotations stay valid rotations.
test_imu.py	7	imu.py protobuf parsers that read gyroscope/accelerometer samples from the ARCore recording, including truncated or garbage input.
test_common_capture.py	11	Reading poses.csv, points.csv and tracking.csv: interpolation between samples, no pose inside a tracking gap, intrinsics, frame-time offsets.
test_common_project.py	6	OpenGL→OpenCV conversion and pinhole projection (points behind or outside the image rejected).
test_relocalize_and_sfm.py	6	Rigid segment correction (relocalize.py) and the Umeyama Sim3 fit used by sfm_colmap.py: recovers a known scale/rotation/translation and returns a proper rotation.
test_process_dataset.py	3	PLY export and transforms.json generation from a synthetic capture.
test_server_api.py	19	FastAPI endpoints with the two job starters replaced by recorders: health, config (API key never returned, mode/GPU switching), upload (required files, depth zip, corrupt zip), status, jobs list, download (404/409/200).
test_server_pipeline.py	12	server/pipeline.py: six steps in order, each with a label and positive timeout, iteration count reaches the training command, Job.to_dict() fields, log truncation; the RunPod command list matches the local pipeline step for step.
test_runpod_client.py	14	server/runpod_client.py with network and subprocesses faked: GraphQL queries and errors, pod creation, SSH-port discovery, wait_for_ready success and timeout, terminate_pod, ssh/scp command construction.

File	Tests	What it checks
`test_apk_checks.py`	20	The APK/manifest checker used by CI/CD: min SDK, package name, permissions, AR feature, processing screen, signing certificate, APK size; the real manifest passes. This checks packaging, not app logic.
`test_sources_compile.py`	1*	Every Python file in the repository compiles. *Parametrised over every `.py` file.

C.2 Test cases

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-1	Pose logging completeness (`CaptureLogger.java`)	No Android unit test exists. The Python reader of `poses.csv` is covered by `test_common_capture.py`.	One pose row per tracked frame with valid position, quaternion and intrinsics	Device (manual)	Not yet automated
TC-2	IQA blur detection (`QualityMonitor.java`)	No Android unit test exists; no recorded on-device test run.	Blur warning shown on blurred frames	Device (manual)	Not yet automated
TC-3	Coverage voxel counting (`CoverageMap.java`)	No Android unit test exists; no recorded on-device test run.	Voxel direction counts drive red/yellow/green dots	Device (manual)	Not yet automated
TC-4	IMU fallback bridging	`test_fallback.py`, `test_common_fallback.py`, `test_imu.py` (36 tests, synthetic data)	Bridged poses land on the end anchor; rotations stay valid; gyro bias removed	Unit	Pass (CI)
TC-5	COLMAP SfM convergence	First room: v3 trained on 574 frames; 1166 gyro-selected frames registered into the v3 model	Frames register; reprojection error <1.0 px	System	Pass (manual): 1108 of 1166 kept; global BA 0.97 $\rightarrow$ 0.88 px

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-6	Sim3 scale	`test_relocalize_and_known_pose` (Umeyama fit on synthetic data); scale checked against a tape measure	Known Sim3 recovered; model distances within 5% of tape	Unit + Accept.	Pass (CI); tape: mean 0.19%, worst 0.29%
TC-7	RunPod client	`test_runpod_client.py` with faked network	Correct GraphQL calls, SSH discovery, timeout raised, terminate call sent	Unit	Pass (CI)
		Live run on a real RunPod pod, including termination on failure	Pod terminated in every path	Integ.	Not yet executed
TC-8	Upload endpoint	`test_server_api.py`: multipart upload, missing file, depth zip, corrupt zip	Files saved, job started; bad uploads rejected cleanly	Integ.	Pass (CI)
TC-9	Status polling	`test_server_api.py`, `test_server_pipeline.py`	Status has the fields the app reads; unknown job is 404	Integ.	Pass (CI)
TC-10	Config switching	`test_server_api.py`: POST then GET `/config`	Mode and GPU switch; unknown mode ignored; API key never returned	Integ.	Pass (CI)
TC-11	End-to-end, first room	Capture `newRoomAttemptSadness` → local pipeline → model; distances vs tape	`splat.ply` produced; distances within 5% of tape	System	Pass (manual) for the local pipeline. App upload → server → RunPod path: not yet executed as a recorded test

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-12	End-to-end, second room	Capture `roomAttemptSadness2` → local pipeline → model	Model produced; held-out SSIM ≥ 0.85	System	Pass (manual): SSIM 0.91, 1% badly placed surfaces. Tape measurement: not yet done
TC-13	Reconstruction quality	Evaluate the v6 model on the 42 held-out frames of the first room (`fair_compare.py`)	PSNR $\geq$ 25 dB (actual: 26.52); SSIM $\geq$ 0.85 (actual: 0.8886)	Accept.	Pass (manual)
TC-14	Processing timeout	Set a step timeout below its run time; run the pipeline	Step killed; job status "failed"; error mentions timeout	Integ.	Not yet executed
TC-15	Pipeline consistency	`test_server_pipeline.py`	Spy steps in order with positive timeouts; RunPod and local pipelines match; called scripts exist	Unit	Pass (CI)
TC-16	Download endpoint	`test_server_api.py`	404 unknown job; 409 unfinished or failed job; file returned when finished	Integ.	Pass (CI)
TC-17	APK and manifest checks	`test_apk_checks.py`	Wrong SDK, package, permission, signer or size reported; real manifest passes	Unit	Pass (CI)
TC-18	Coordinate conventions	`test_common_project.py`, `test_common_capture.py`	GL→CV flips y and z; projection and pose interpolation correct	Unit	Pass (CI)
TC-19	Dataset export	`test_process_dataset.py`	PLY written with header; `transforms.json` built	Unit	Pass (CI)

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-20	Source compilation	`test_sources_compile.py`	Every Python file compiles	Static	Pass (CI)

Appendix D

Repository and Contribution Evidence

D.1 Repository Structure

Path	Contents
`googleARCore/samples/hello_ar_java/`	Android capture app; Java sources in `app/src/main/java/.../helloar/`
`HelloArActivity.java`	main AR activity, recording, settings
`CaptureLogger.java`	sensor data recording
`QualityMonitor.java`	live IQA analysis
`CoverageMap.java`, `CoverageOverlayView.java`	voxel coverage tracker and dot overlay
`ProcessingActivity.java`	upload and progress UI
`GalleryActivity.java`, `ScanDetailActivity.java`	saved scans
`src/arcore3dgs/`	reconstruction pipeline
`fallback.py`, `imu.py`, `relocalize_pnp.py`	IMU tracking-gap bridging and post-gap re-alignment
`prepare.py`, `select_sharp.py`	keyframe extraction; gyro-aware frame selection
`sfm_colmap.py`, `export_nerfstudio.py`	COLMAP SfM + Sim3 alignment; Nerfstudio export
`mvs_init.py`, `structure.py`	dense MVS initialisation; structural layer evaluation tools
`fair_compare.py`, `fit_diag.py`	
`server/`	processing server: `main.py`, `pipeline.py`, `runpod_client.py`, `remote_gpu.py`

Path	Contents
`.github/workflows/`	CI/CD: `ci.yml`, `cd.yml`, `docker-gpu.yml`
`scripts/`	launchers: `start_server.bat`, `view_splat.bat`, job tools
`tests/`	pytest suite (12 files + `conftest.py`)
`prototype/web-ui/`	web UI prototype
`docs/progress/`	`TODO.md`, `Training_Analysis.md`, `Problems_and_Solutions.md`
`.env`	API keys (gitignored)

D.2 Commit History Summary

The repository contains 133 commits (`git shortlog -sne -all`): Muhammad Umair 85, Zaid Fareed 26, Muhammad Tayyab 20 (as "BlazingBeam"), and 2 GitHub Actions bot commits. The counts reflect commit granularity rather than effort: Umair commits each small change, Tayyab commits whole pipeline stages in large batches. Owned modules are listed in Table 8.1. Key commit categories:

- **Muhammad Umair**: processing server, RunPod integration, upload and gallery screens, AR activity and coverage overlay work, web UI prototype, proposal, report and documentation.
- **Zaid Fareed**: CI/CD workflows (`ci.yml`, `cd.yml`), APK checker, the pytest suite, pytest and ruff configuration, pull request template, trainer fixes and ARKitScenes experiments.
- **Muhammad Tayyab**: reconstruction pipeline (`src/arcore3dgs/`), capture logging and live IQA in the app, training analysis, v3–v6 models, tape-measure validation and the structural layer.

D.3 Repository Overview

D.4 Branch Strategy

- `master`: the integration branch, and the default branch.

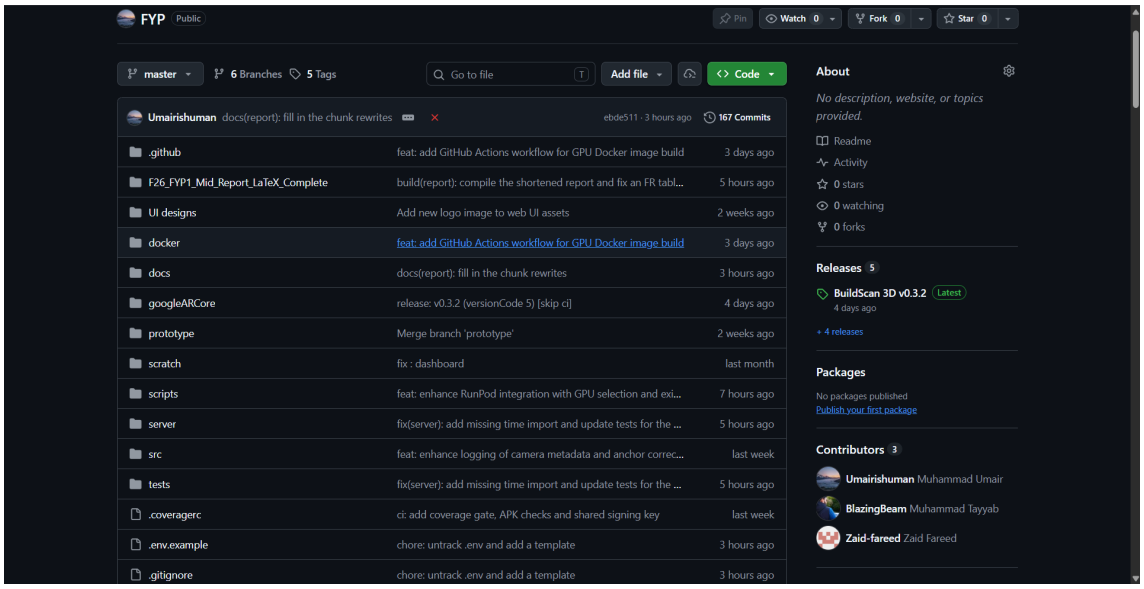


Figure D.1: Repository root on GitHub. The contributor list confirms the three team members and the GitHub handles used in the commit history: `Umairishuman` (Muhammad Umair), `BlazingBeam` (Muhammad Tayyab) and `Zaid-fareed` (Zaid Fareed). The top-level layout matches Table 8.2.

- Feature branches (e.g. `ci/github-actions`, `ci/pipeline-updates`, `exp/trainer-arkitscenes`) short-lived branches for specific work items.
- Branches are merged locally rather than through pull requests. The pull request template exists and CI runs on feature branches, but the merges themselves were not raised as PRs, so the "Pull request" column in Figure D.2 is empty. Adopting PR-based merges is an action for FYP-2.

D.5 CI/CD Evidence

The GitHub Actions CI pipeline runs on pushes to non-`master` branches and on pull requests; direct pushes to `master` are covered by the CD workflow instead.

1. **Python checks**: ruff lint + pytest on Python 3.10, 3.11, 3.12.
2. **Web build**: Node.js build of the web UI (placeholder for FYP-2).
3. **Android build**: Gradle APK build.
4. **Hygiene**: File size limits, secrets detection, workflow lint.

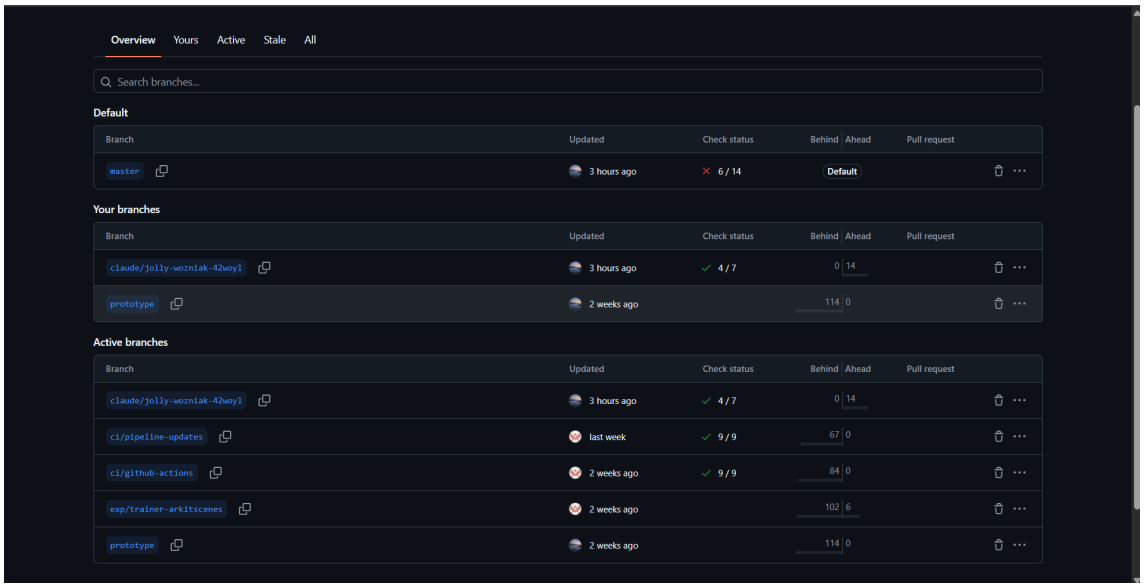


Figure D.2: Branch list with each branch's check status and its ahead/behind count against `master`. Feature branches carry passing checks; the failing status on `master` is the hygiene job described in Section D.5.

The CD pipeline triggers on push to master (build artifacts) and on version tags (GitHub Release with APK and web bundle).

D.6 Task Tracking

Development progress is tracked in `docs/progress/TODO.md`, which serves as the project's task log with dated entries, status markers, and links to relevant commits and analysis documents.

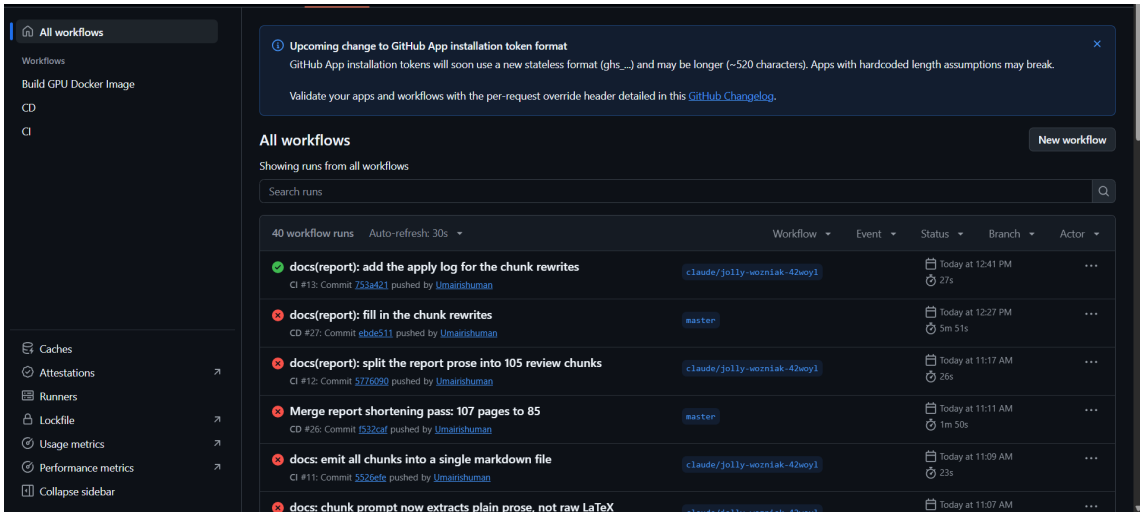


Figure D.3: GitHub Actions run history. CI passes on the feature branches; the failing runs on `master` are the CD workflow's hygiene job, which rejects a tracked `.env`. That file has since been untracked and ignored, and the key it held must still be rotated (NFR-S2, recorded as not met).

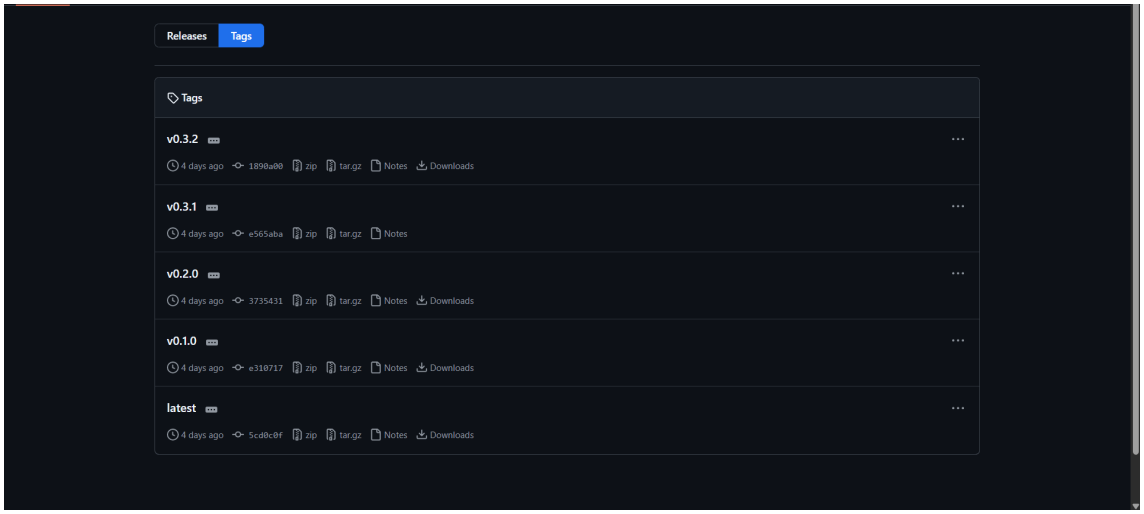


Figure D.4: Release tags produced by the CD workflow. Each version tag builds a GitHub Release carrying the signed APK and the web bundle, so every tagged build is reproducible from its commit.

Appendix E

Additional Design Diagrams

The main design diagrams are in the body of the report: the use case diagram (Figure 4.1), the system architecture (Figure 5.1), the job state machine (Figure 5.2), the data flow (Figure 5.3) and the upload–process–download sequence (Figure 5.4). This appendix adds five further diagrams and one screenshot of the running system. Each diagram is drawn from the code it names.

E.1 Pipeline Step Sequence

Figure E.1 expands the six processing steps of Section 5.8 with the per-step timeout enforced by the watchdog and, where one was recorded, the time the step actually took on the development RTX 3070.

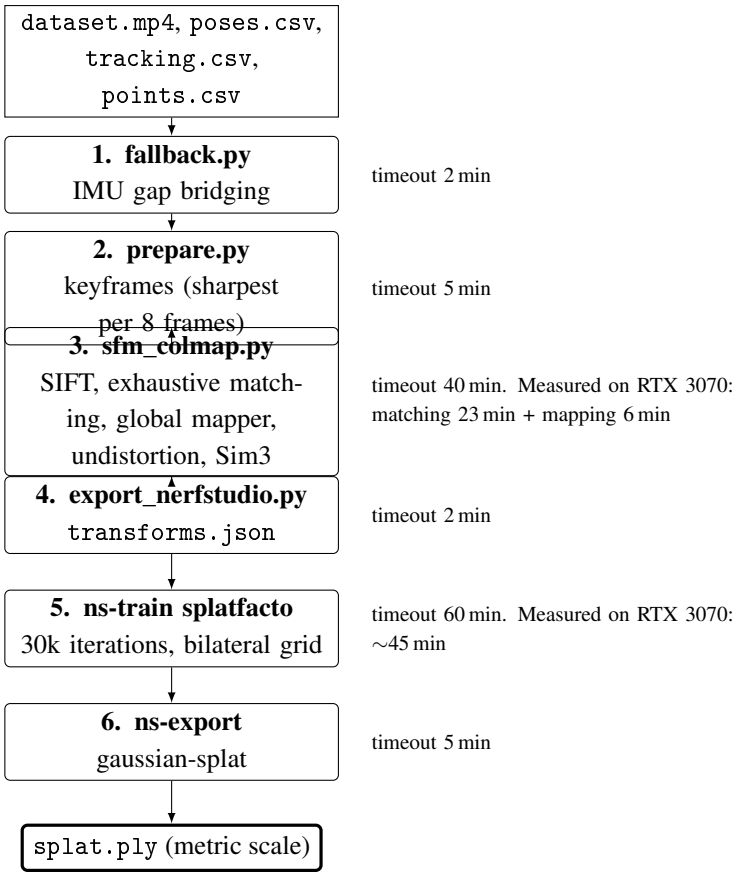


Figure E.1: The six pipeline steps and their timeouts (`STEPS` in `server/pipeline.py`). The measured end-to-end time on the development RTX 3070 is ∼75 min; no cloud-GPU time has been measured yet.

E.2 Android App Architecture

Figure E.2 shows how the capture app is put together: which class owns which output file, how the activities hand off to one another, and which files stay on the phone rather than being uploaded.

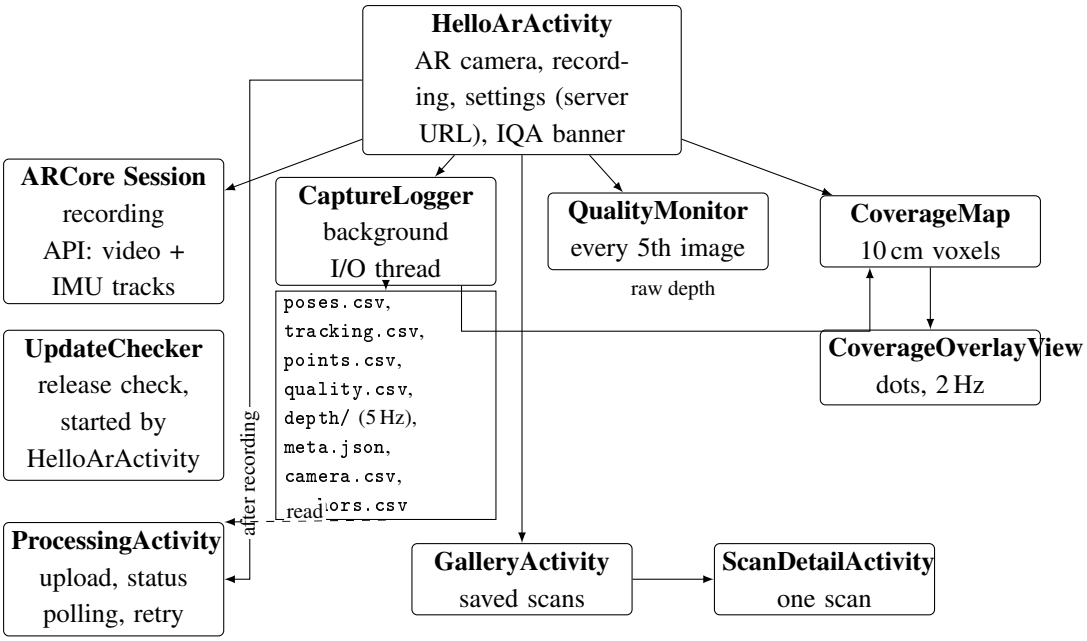


Figure E.2: Android capture app (`googleARCore/samples/hello_ar_java/.../helloar/`). Arrows from `HelloArActivity` show ownership or navigation (`Intents`); `CaptureLogger` feeds raw depth to `CoverageMap`. `ProcessingActivity` uploads six files plus a zipped `depth/` folder; `camera.csv` and `anchors.csv` stay on the phone.

E.3 Sim3 Alignment

Figure E.3 shows how the COLMAP reconstruction is brought into ARCore's metric frame. This is the step that gives the model its real-world scale, so every measurement in Table 7.6 depends on it.

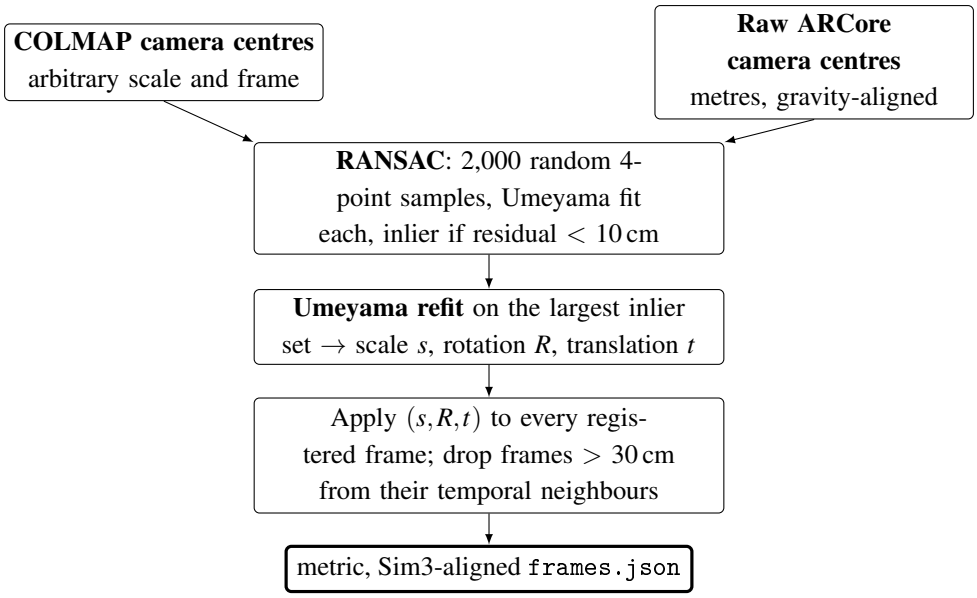


Figure E.3: Sim3 alignment of the COLMAP reconstruction to ARCore's metric frame (`sfm_colmap.py`, export stage; Umeyama fit unit-tested in `test_relocalize_and_sfm.py`). Because ARCore re-anchors silently, only its largest self-consistent segment ends up as inliers. Inlier median residual: about 2.5 cm. The resulting scale is validated against a tape measure (Table 7.6).

E.4 Coverage Map

Figure E.4 shows how the live overlay decides a voxel's colour, which is the guidance the user acts on while walking the room.

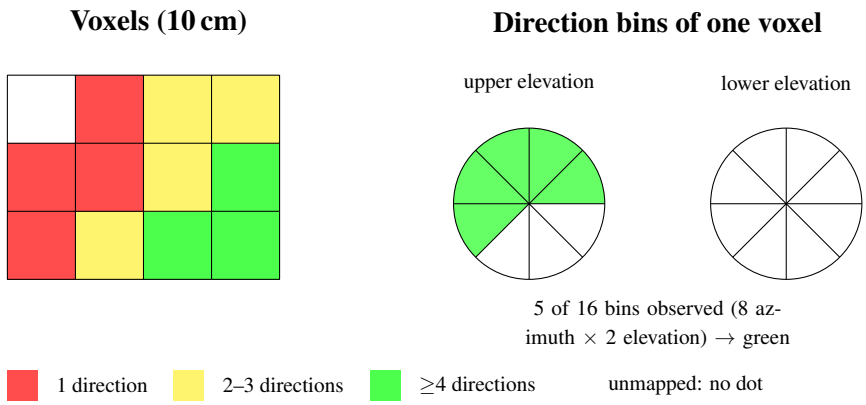


Figure E.4: Live coverage map (`CoverageMap.java`, `CoverageOverlayView.java`). ARCore raw depth (5 Hz, confidence $\geq$128, up to 5 m) is accumulated in 10 cm voxels; each voxel records from which of 16 directions it was seen. Visible voxels are drawn as dots, coloured by direction count, with at most 5,000 voxels projected and 2,000 dots drawn, refreshed at 2 Hz. The grid colouring is illustrative.

E.5 Domain Model

Figure E.5 draws the entities listed in Section 5.6 and the relationships between them. The attributes shown are the fields the code actually carries, so the diagram doubles as a map of what is written to disk at each stage.

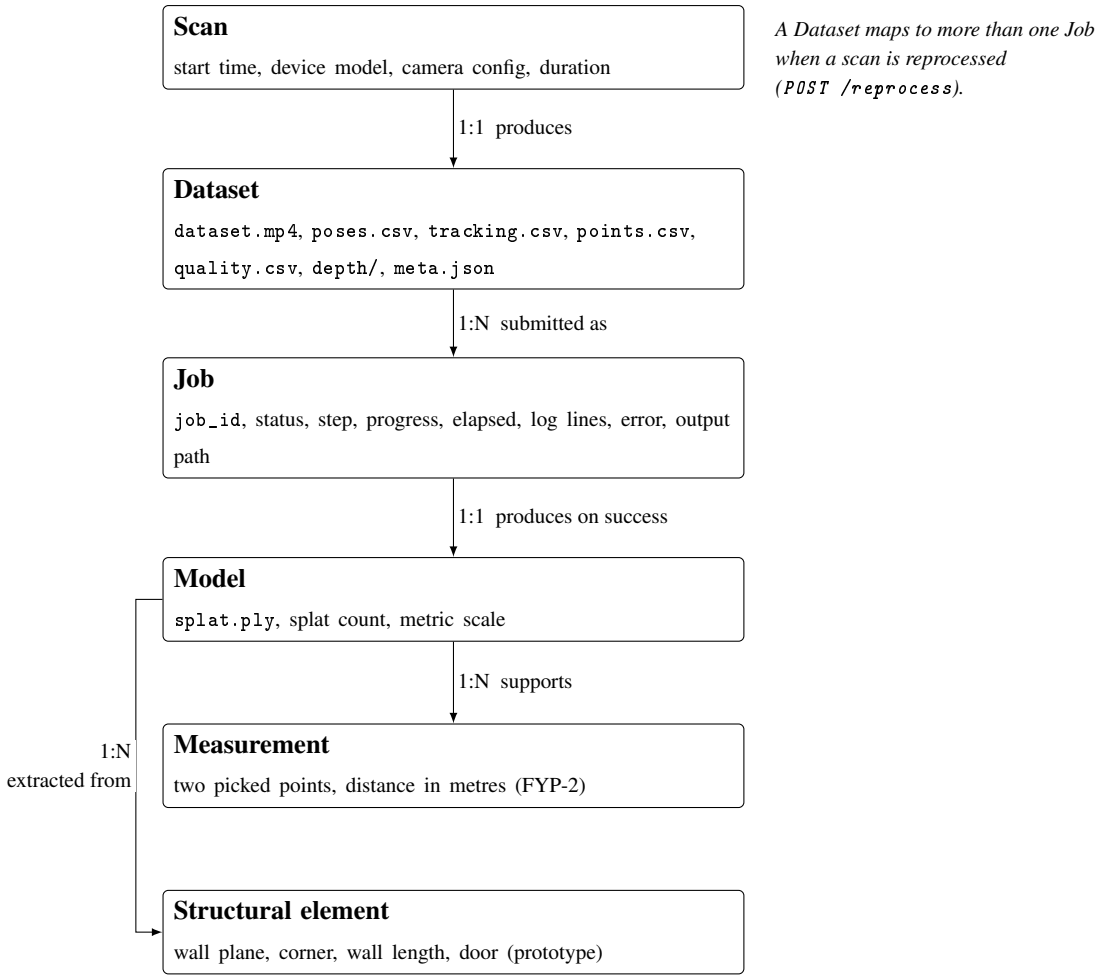


Figure E.5: Domain model. A **Scan** produces exactly one **Dataset**; a Dataset can be submitted as more than one **Job**, because a scan may be reprocessed after a pipeline change; a Job that succeeds produces one **Model**. **Measurement** and the web-facing part of **Structural element** are FYP-2 work, drawn here because the metric scale they depend on is already established by the Sim3 step in Figure E.3. Entity fields follow `Job` in `server/pipeline.py` and the capture files written by `CaptureLogger.java`.

E.6 Capture Application in Use

Figure E.6 shows the capture app recording, with the quality and coverage features of Figure E.4 running live on the phone.

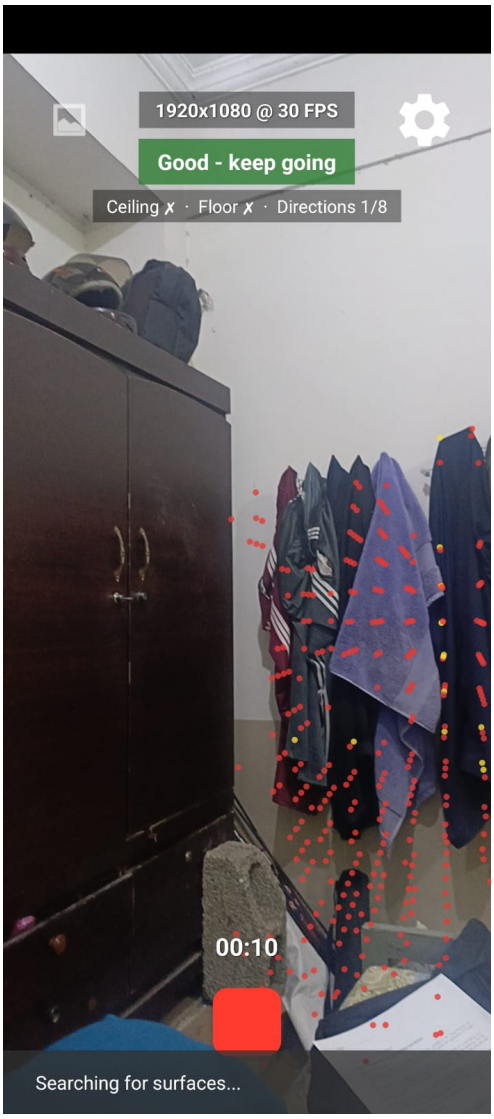


Figure E.6: The Android capture app mid-recording on the realme device. The header confirms the capture format (1920×1080 at 30 FPS, FR-M1). The green banner is the live IQA verdict from `QualityMonitor.java`; the row beneath it reports coverage state from `CoverageMap.java` — ceiling and floor not yet seen, and the current voxel observed from 1 of 8 azimuth directions. The red and yellow dots are the coverage overlay drawn by `CoverageOverlayView.java`: red marks surfaces seen from a single direction, yellow from two or three. The elapsed-time counter and the stop control complete the recording UI, and "Searching for surfaces" is ARCore still building its plane estimate early in the session.

Appendix F

Meeting and Progress Evidence

F.1 Supervisor Meetings

The team held **two** meetings with the supervisor, Mr. Furqan Nasir, before the mid-evaluation. Their dates were not recorded at the time, so they are listed in order rather than by date. No formal minutes were taken, and no decision in this report is presented as having been approved by the supervisor.

Meeting	Agenda	Attendees
First meeting	Showing the work completed to date; the techniques used; clarifying the expectations for the mid-evaluation.	Project team and supervisor
Second meeting	Showing the work completed to date; the techniques used; clarifying the expectations for the mid-evaluation.	Project team and supervisor

The dated record of the work itself is the progress log below and the commit history (Chapter 8).

F.2 Progress Log

Detailed development progress is maintained in `docs/progress/TODO.md`. Key milestones:

- **ARCore capture app**: Basic recording → pose sync validation → IQA → coverage overlay → upload UI. Incremental feature additions documented with timestamps.

- **Reconstruction pipeline**: Best custom-trainer model on ARCore-derived poses (`gs_colmap_nc`, 22.67 dB / 0.782 held-out) → image-based COLMAP poses (training-view fit 21.9 → ∼28 dB; held-out SSIM 0.78 → 0.86) → v3 (26.04 dB / 0.8756 on the 42-frame held-out set) → gyro-aware frame selection (+0.26 dB / +0.009 SSIM) → dense MVS initialisation (+0.21 dB / +0.003 SSIM) → v6 (26.52 dB / 0.8886). Each improvement documented with before/after metrics.
- **Quality debugging**: Two-week root-cause analysis documented in `Training_Analysis.md` covering 6 identified causes of degenerate splats, with per-cause experiments and metrics.
- **Server and cloud**: FastAPI server → local pipeline runner → RunPod integration → guaranteed pod lifecycle. Problem-solution pairs documented in `Problems_and_Solutions.md`.

F.3 Problems and Solutions Summary

25 problems encountered during development are documented in `docs/progress/Problems_and_Soluti`
The most significant:

1. ARCore silent re-anchoring causing pose discontinuities (solved: anchor logging + IMU fallback)
2. COLMAP lens distortion not matching ARCore intrinsics (solved: self-calibration)
3. Degenerate splat geometry from custom training recipe (solved: adopted splatfacto)
4. 30 FPS input overwhelming COLMAP matching (solved: keyframe selection, one sharp frame per window)
5. Motion blur in training frames, 6–12 px median (solved for v4–v6: gyro-aware frame selection, +0.26 dB / +0.009 SSIM)
6. GPU memory overflow on the 8 GB RTX 3070 with >1000 full-resolution frames (solved: skip page-locked image caching above 4 GB; default densification thresholds)

Bibliography

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," in *ACM Transactions on Graphics (SIGGRAPH)*, vol. 42, no. 4, 2023.

[2] J. Kim, J. Park, D. Shin, and Y. M. Kim, "Pocketgs: On-device 3d gaussian splatting training and rendering," *arXiv preprint arXiv:2503.10435*, 2025.

[3] Z. Zheng *et al.*, "Vigs-slam: Visual-inertial gaussian splatting slam," in *arXiv preprint arXiv:2411.00483*, 2024.

[4] D. Chen, H. Li, W. Ye, Y. Wang, W. Xie, S. Zhai, N. Wang, H. Liu, H. Bao, and G. Zhang, "Pgsr: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.